Sonderdruck aus
Datenverarbeitung im Recht Band 5, Heft 4/1976
J. Schweitzer Verlag Berlin

© Copyright 1976 by J. Schweitzer Verlag Berlin. Alle Rechte, insbesondere die der Übersetzung in fremde Sprachen, vorbehalten.

L. Thorne McCarty

Reflections on TAXMAN: An Experiment in Artificial Intelligence and Legal Reasoning*

Table of Contents

I. Introduction
II. Some Fundamentals of Corporate Reorganization Law
III. Some Fundamentals of Semantic Information Processing
IV. Implementations
A. States and Events
B. Description Mechanisms
C. Analysis Mechanisms
V. Results
VI. Feasible Extensions
VII. Conclusion

I. Introduction

This article addresses – in a highly unorthodox way – a series of questions which are surely among the most orthodox that can be found anywhere in the legal and jurisprudential literature. The questions are concerned primarily with legal concepts: What is the structure of a legal concept? What is the process by which legal concepts are transformed and modified? What are the arguments that lawyers characteristically make with respect to legal concepts? These are the kinds of questions that the Analytical Jurists once treated as essential, and the Realists dismissed as irrelevant; the kinds of questions that Cardozo ascribed to the Method of Philosophy in the judicial process, as opposed to the Method of Sociology[1]. In short, I will be raising here some of the classical problems of legal reasoning, in their most classical form.

But my approach to these problems will be anything but classical. I will describe in the following pages a computer program I have written which models certain aspects of the conceptual structures which occur in a specific area of the law: namely, the taxation of corporate reorganizations, as set forth in Subchapter C of Chapter 1 of the Internal Revenue Code of 1954[2]. The program – dubbed mnemonically TAXMAN – is capable of performing a very rudimentary form of "legal reasoning": it can be given a "description" of the "facts" of a corporate reorganization case, and it can then develop an "analysis" of these facts in terms of several conclusory legal "concepts". I will describe in considerable detail how this program operates, what its limitations are, and how it might be revised and rewritten to achieve a somewhat higher level of sophistication[3]. In this way – by

Copyright © 1977 by L. Thorne McCarty (see page 373).

* Part One of a two-part series of articles

1 *B. Cardozo,* The Nature of the Judicial Process (1921).

2 Int. Rev. Code of 1954, §§ 354–56, 358, 361–62, 368.

3 The TAXMAN program was written in 1972–73, and it was first discussed in a paper

looking carefully at both the adequacies and inadequacies of a particular formal model of the concepts that occur in a particular area of the law – I hope to be able to develop some insights about the structure and function and dynamics of legal concepts generally[4].

I realize, of course, that the use of a formal computer model may be a strange and disquieting prospect to most members of a legal audience. There is a suggestion here, perhaps, of a "mechanical jurisprudence" run amok – a computer sitting as a judge, hearing cases, rendering decisions. But this is certainly not the purpose of the present work. Nor, under any conceivable circumstances, is it a plausible consequence. In fact, the present article exposes most clearly the limitations of any such automated judicial decisionmaking. The TAXMAN system in its current version accomplishes only the most simple-minded tasks: it classifies a case under a statutory rule as a Type B, a Type C, or a Type D reorganization, and even this it

---

presented at a workshop on Computer Applications to Legal Research and Analysis, Stanford Law School, April 28–29, 1972. *McCarty,* Interim Report on the TAXMAN Project: An Experiment in Artificial Intelligence and Legal Reasoning, in Artificial Intelligence Techniques in Legal Problem Solving, June 1, 1973 (mimeo, Stanford Law School). This early paper, though unpublished, was widely circulated among interested persons in the field, and it has itself been the subject of some occasional discussion in the published literature. The most extensive analysis appears in *P. Slayton,* Electronic Legal Retrieval 18–19, 25 (Canadian Department of Communications, 1974); *P. Slayton,* Radical Computer Use in Law 67–68 (Draft Report prepared for the Canadian Department of Communications, June, 1974); *Slayton,* Electronic Legal Retrieval – The Impact of Computers on a Profession, 14 Jurimetrics J. 29, 36 n. 7, 39 (1973); *Slayton,* Computers and the Law – An Uneasy Trial Marriage, 1 In Search/En Quete 14 (1974). Additional references to TAXMAN appear in Boyd, Law in Comupters and Computers in Law: A Lawyer's View of the State of the Art, 14 Ariz. L. Rev. 267, 286 n. 107 (1972); *Slayton,* New Approaches to Legal Study, 1 Dalhousie L. Rev. 163, 163 n. 3 (1973); *Bauer-Bernet,* Effect of Information Science on the Formation and Drafting of Law, 14 Jurimetrics J. 235, 245 n. 14 (1974); *Popp & Schlink, Judith:* A Computer Program to Advise Lawyers in Reasoning a Case, 15 Jurimetrics J. 303, 313–14 (1975); Note, Computer Simulation and Gaming: An Interdisciplinary Survey with a View Toward Legal Applications, 24 Stan. L. Rev. 712, 728 n. 114 (1972); *Mackaay,* Book Review, 15 Jurimetrics J. 327, 328 (1975). The present article, however, contains the first full-length description of the program.

4 The most important precursor to the present article is *Buchanan & Headrick,* Some Speculation About Artificial Intelligence and Legal Reasoning, 23 Stan. L. Rev. 40 (1970). This paper identified several components of the legal reasoning process, "(1) finding conceptual linkages in pursuing goals, (2) recognizing facts, (3) resolving rule conflicts, and (4) finding analogies", id. at 53, and compared these carefully to some of the components of the more advanced and experimental computer programs in existence at the time, especially the Heuristic *Dendral* program for solving mass spectrometry problems in organic chemistry, id. at 53–60. The paper urged that serious research be undertaken to apply these computer science methods to legal problem-solving, and speculated that such research "could lead both to a greater understanding of the legal reasoning process and to the design of machine methods for performing parts of it", id. at 41. To the best of my knowledge, the TAXMAN system was the first attempt to implement in a functioning computer program some of these suggestions. Having actually carried out this implementation, my own emphases and my own conclusions differ somewhat from those of Buchanan and Headrick, but their paper remains as the most important initial statement of the approach. More recently, there have been several

does with difficulty[5]. There are several feasible extensions to the current system – extensions to encompass a broader range of factual situations and a broader range of legal concepts – but these feasible extensions will be shown to have their own severe limitations as well[6]. The TAXMAN system should thus clarify further, if any further clarification is needed, the reasons why a fully mechanical jurisprudence is unattainable[7].

Speaking more realistically, however, there are two main purposes behind the present work. The first of these is a severely practical one, and of quite modest scope. Despite its limitations, the TAXMAN system provides a number of techniques that should be useful in automating the more mundane aspects of legal research and analysis. The existing systems for computer-aided research and analysis in the law take two forms: the first depends on an exhaustive search through the full text of a body of legal materials, and a retrieval of documents by key-words or combinations of key words[8]; the second depends on a sequence of pre-programmed questions and answers about a legal problem that is designed to terminate in a legal conclusion, in much the same style as a program for computer-aided instruction (CAI)[9]. But both of these approaches have limitations, as one might suspect. The TAXMAN system, with its computer representations of actual legal concepts, suggests a more structured, more flexible, potentially more powerful alternative, although it remains to be seen how practical and efficient this alternative will be[10]. At the very least, the prospects seem promising enough to justify a substantial amount of additional research.

The second and more interesting purpose behind the present work is a theoretical one. Whatever its practical applications, the TAXMAN system provides an important tool for the development of our theories about legal reasoning, or so I wish to claim. It does this by providing a means of formalizing these theories, an idea that requires some elaboration in the present context. I take it as a premise that a great many of the classical jurisprudential problems are tied to problems about the uses of language, or the uses of abstract concepts generally, in the regulation of human affairs. Thus *Dworkin* bases his analysis of judicial discretion to a large extent on his "logical distinction" between "legal rules" and "legal principles"[11]; *Hart* and

projects undertaken along similar lines. The most sophisticated of these is *Meldman,* A Preliminary Study in Computer-Aided Legal Analysis, Project MAC Report No. TR-157 (PhD. Dissertation, Mass. Inst. of Technology, 1975), which presents a detailed design of a program to assist in the analysis of a simple assault and battery case, but reports no actual implementation. See also *Stamper,* The *Legol* Project and Language, in Proceedings, Datafair Symposium, 1973 at 263 (British Computer Soc'y, London); *Popp & Schlink, Judith:* A Computer Program to Advise Lawyers in Reasoning a Case, 15 Jurimetrics J. 303 (1975); Note, A Computer Method for Legal Drafting Using Propositional Logic, 53 Texas L. Rev. 965 (1975).

5 See pp. 338/339, 353–358 infra.

6 See pp. 359/360 and notes 126–34 infra.

7 For a furthes discussion, see Part Two of this article.

8 See pp. 367/368 and notes 156–59 infra.

9 See pp. 368–370 and notes 160–63 infra.

10 See pp. 371 infra.

11 *Dworkin,* The Model of Rules, 35 U. Chi. L. Rev. 14, 22–40 (1967).

Fuller devote a portion of their debate on legal positivism to a dispute about the "core" and the "penumbra" and the "open texture" of legal concepts[12]; *Levi* describes the law as the prime example of a "moving classification system", and sees in this description an explanation for many of the perplexing characteristics of the legal process as a whole[13]. In these writings and others, then, there are many valuable insights about the structure and function and dynamics of legal concepts, and many illuminating examples[14]. But taken as a whole, the jurisprudential literature is notoriously imprecise about these linguistic and conceptual ideas: the conceptual structures themselves are only vaguely defined, and vaguely distinguished from one another; the functional considerations are emphasized, often persuasively, but impressionistically; the dynamics of conceptual change appear only as suggestive metaphors.

The TAXMAN system leads, potentially, to an entirely different approach. It adds a strong dose of precision and rigor to these discussions of linguistic and conceptual problems. In the TAXMAN system, as we shall see, the critical task is to describe the concepts of corporate reorganization law in ways that can be represented in computer programs, and this requires a degree of explicitness about the structure of these concepts that has never previously been attempted. When we describe concepts in this way, of course, we articulate theories about them; when we run the computer programs that embody these concepts, we test out the implications of our theories. Used in this fashion, the computer is the most powerful tool for expressing formal theories and spinning out their consequences that has ever been devised.

We can see this point most clearly, I think, if we examine briefly the contemporary traditions in linguistics and cognitive psychology, two areas with obvious connections to the present work. I assume that most lawyers are generally aware of the kinds of ideas that have been advanced in these subjects – the revolutions wrought by *Chomsky* and others in linguistics[15], and by *Piaget* and others in cognitive psychology[16] – since these ideas have by now worked their way fairly thoroughly

12 *Hart,* Positivism and the Separation of Law and Morals, 71 Harv. L. Rev. 593, 606–15 (1958); *Fuller,* Positivism and Fidelity to Law – A Reply to Professor Hart, 71 Harv. L. Rev. 630, 661–69 (1958); *H. L. A. Hart,* The Concept of Law 121–50 (1961); *L. Fuller,* The Morality of Law 224–32 (rev. ed. 1969).

13 *E. Levi,* An Introduction to Legal Reasoning 1–8, 102–04 (1949).

14 One of the best collections of materials of this sort is *W. Bishin & C. Stone,* Law, Language and Ethics: An Introduction to Law and Legal Method (1972), in which the law is studied as "a special but not unique effort of the human mind to make 'reality' comprehendable and manageable", id. at vii.

15 See, e. g., *N. Chomsky,* Syntactic Structures (1957); *N. Chomsky,* Aspects of the Theory of Syntax (1965); *N. Chomsky,* Language and Mind (1968); *N. Chomsky,* Reflections on Language (1976).

16 See, e.g., *J. Piaget,* The Child's Conception of Physical Causality (1930); *J. Piaget,* The Origins of Intelligence in Children (1952); *J. Piaget,* The Child's Conception of Number (1952); *J. Piaget & B. Inhelder,* The Child's Conception of Space (1956); *B. Inhelder & J. Piaget,* The Early Growth of Logic in the Child (1964); *J. Flavell,* The Developmental Psychology of Jean Piaget (1963). Unlike the situation in linguistics, however, where the work of Chomsky clearly dominates all current schools of thought, the revolution in

into the popular literature[17]. Both disciplines, in opposition to their behaviorist predecessors, posit abstract mental structures as explanations of basic linguistic and psychological facts. In linguistics, the proposed mental structures are syntactic and semantic in nature, and they puport to organize and explain our intuitions about the grammaticality of various sentences; in cognitive psychology, where the schools of thought are more varied and diffuse, the proposed structures may be perceptual schemas, or memory mechanisms, or devices for acquiring and abstracting concepts. But what is especially striking about the abstract structural models that appear in these disciplines today is the fact that they are so closely tied to computational structures – that is, to the kinds of structures that are most naturally represented by computer programs. *Herbert Simon* has remarked that modern transformational linguistics and modern information processing psychology have their proximate origins in two papers presented together at a computer-oriented symposium at M.I.T. in 1956[18], and whether or not this historical observation is entirely accurate, it certainly captures the flavor of much of the contemporary work in these fields. There are many examples in the recent literature: semantic theories formulated and tested within a computer system that conducts a simple dialogue in English[20]; theories of associative memory written in the form of programs, and validated by experimental data from human subjects[21]. In these areas at least, the computer has proven itself to be one of the ideal modes of expression for linguitic and psychological ideas.

So, too, with TAXMAN. But in taking this approach, one must proceed with a great deal of sophistication about the uses and abuses of formal models. A formal model is necessarily a simplification of reality; it omits details, by design, which in many contexts might be crucial; and so, by design, it will always be inadequate in one or more respects. And yet the simplification inherent in a formal model is also the

---

cognitive psychology has had a much more heterogeneous origin, and I have cited Piaget here primarily because his work is most likely to be familiar to the general reader. A list of other influential works would include, e.g., *J. Bruner, J. Goodnow & G. Austin,* A Study of Thinking (1956); *G. Miller, E. Galanter & K. Pribram,* Plans and the Structure of Behavior (1960); *U. Neisser,* Cognitive Psychology (1968).

17 These ideas have even appeared occasionally in the legal literature. See, *Hermann,* A Structuralist Approach to Legal Reasoning, 48 S. Cal. L. Rev. 1131 (1975) (an application of the theories of *Chomsky, Piaget* and *Levi-Strauss*); Comment, Jurisprudence and the Nature of Language: Contrasting Views of Hart and Chomsky, 42 Wash. L. Rev. 847 (1967).

18 *H. Simon,* The Sciences of the Artificial 47 (1969). The two papers were *Newell & Simon,* The Logic Theory Machine, 2 I.R.E. Transactions on Information Theory 61 (1956), and *Chomsky,* Three Models for the Description of Language, 2 I.R.E. Transactions on Information Theory 113 (1956).

20 See, e.g., *T. Winograd,* Understanding Natural Language (1972); *Schank,* Conceptual Dependency: A Theory of Natural Language Understanding, 3 Cognitive Psychology 552 (1972); Woods, Transition Network Grammars for Natural Language Analysis, 13 Comm. of the Ass'n for Computing Mach. 591 (1970).

21 See, e.g. *D. A. Norman, D. Rumelhart & The LNR Research Group, Explorations in Cognition (1975); J. R. Anderson & G. Bower,* Human Associative Memory (1973); *Quillian,* Semantic Memory, in Semantic Information Processing 227 (*M. Minsky* ed. 1968).

source of its power and utility: it will often lead us to insights that would otherwise be obscured, overwhelmed by the complexity of our data. *Chomsky* has stated this point succinctly and accurately in the preface to his earlier work:

> Precisely constructed models for linquistic structure can play an important role, both negative and positive, in the process of discovery itself. By pushing a precise but inadequate formulation to an unacceptable conclusion, we can often expose the exact source of this inadequacy and, consequently, gain a deeper understanding of the linquistic data. More positively, a formalized theory may automatically provide solutions for many problems other than those for which it was explicitly designed. Obscure and intuition-bound notions can neither lead to absurd conclusions nor provide new and correct ones, and hence they fail to be useful in two important respects[22].

One moral of this story, however, is that both the negative and the positive roles of formal models must be carefully attended to in any piece of research. In particular, the unacceptable conclusions must not be ignored, but must be exploited systematically for the insights they can yield.

This is precisely the strategy I have attempted to follow. Part One of this article thus explores the positive role of the TAXMAN system, pushing it to its limits. The formal model here comes from a branch of computer science devoted to the study of what is called *artificial intelligence*[23]. The model is designed to represent linguistic and conceptual information in ways that are arguably similar to the representations of this kind of information in the human mind. The techniques used, sometimes referred to as techniques of *semantic information processing*[24],

---

22 *N. Chomsky,* Syntactic Structures 5 (1957).

23 As the name suggests, the field of artificial intelligence (often abbreviated: AI) is concerned with the problem of designing machines which exhibit some form of "intelligent" behavior. This would include machines which play chess, e.g., *Greenblatt, Eastlake & Crocker,* The Greenblatt Chess Program, Proceedings of the Fall Joint Computer Conf. 1967, at 801 (Am. Fed'n of Information Processing Societies, Montvale, N. J.); prove mathematical theorems, e.g., *Bledsoe, Boyer & Henneman,* Computer Proofs of Limit Theorems, 3 Artificial Intelligence 27 (1972); solve mass spectrometry problems in organic chemistry, e.g., *Buchanan, Sutherland & Feigenbaum,* Rediscovering Some Problems of Artificial Intelligence in the Context of Organic Chemistry, in 5 Mach. Intelligence 253 (*B. Meltzer & D. Mitchie,* eds. 1970); and many others. There is no doubt that the name "artificial intelligence" was chosen partly because of the provocative stance it takes on the question – "Can a Machine Think?" – that was raised in the early article by *Turing,* Computing Machinery and Intelligence, 59 Mind 433 (1950). On the unfortunate connotations of this choice of words, see, *H. Simon,* The Sciences of the Artificial 4 n. 1 (1969): "At any rate, 'artificial intelligence' seems to be here to stay, and it may prove easier to cleanse the phrase than to dispense with it. In time it will become sufficiently idiomatic that it will no longer be the target of cheap rhetoric." For a collection of some of the earlier work in the field, see, Computers and Thought (*E. Feigenbaum & J. Feldman,* eds., 1963); Semantic Information Processing (*M. Minsky,* ed., 1968); Representation and Meaning: Experiments with Information Processing Systems (*H. Simon & L. Siklossy,* eds., 1972). For a sharp critique, see *Weizenbaum,* Computer Power and Human Reason (1976); *Dreyfus,* What Computers Can't Do: A Criticism of Artificial Reason (1972).

24 See, e.g., Semantic Information Processing (*M. Minsky,* ed., 1968). I use the term here to indicate a subfield within the broader field of artificial intelligence, and to indicate also a general evolutionary trend in the field of artificial intelligence as a whole. As a subfield, semantic information processing is more closely tied to the work in linguistics and cognitive psychology, supra notes 20 and 21, than it it to the work on game-playing,

are very much a part of the current paradigm in computer science, as well as the current paradigm in linguistics and cognitive psychology, mentioned above. The question then is: How adequate is this paradigm? How adequately does it explain the familiar facts of legal reasoning? Specifically, how well does it handle the taxation of corporate reorganizations? It will become clear, I think, that the TAXMAN paradigm has considerable power, but that it also has several serious deficiencies. Part Two of this article will then discuss these deficiencies further, and suggest some modifications. In the process, Part Two ought to shed light in two directions: first, the detailed work with the formal computer models should generate some ideas about the problems of legal reasoning; and second, the detailed analysis of legal reasoning should generate some ideas about the theoretical models themselves, and how they might be improved.

In many respects Part Two of this article is the more interesting, but it would be premature to discuss it further at this point: the method I have outlined requires that we immerse ourselves first in the details of the current version of TAXMAN. I have organized Part One to accomplish this as painlessly as possible. The two sections following this Introduction are intended to provide the necessary background for anyone who knows nothing about either the legal or the technical foundations of the work. Section II thus presents an elementary introduction to the taxation of corporate reorganizations, and Section III presents an elementary introduction to the basic ideas of semantic information processing. Neither of these sections is intended to provide enlightenment to an expert in the field, of course, but the expert reader might find it helpful to skim through these sections quickly to provide an orientation and to note any idiosyncratic usages that might appear there. With this background, Section IV presents a description of the current implementation of TAXMAN, first a general overview, and then a more detailed look at several important components of the system. Section V presents results – the actual performance of TAXMAN in analyzing a corporate reorganization case – and Section VI outlines several extensions of the program which seem feasible within the present state of the art. Once these feasible extensions are thoroughly explored, and the capabilities of the current paradigm are thoroughly understood, it will be possible to move on to Part Two, and some further speculations.

---

theorem-proving and problem-solving programs, supra note 23. But even within the game-playing programs themselves, the term indicates a shift in emphasis: thus the earlier programs were characterized primarily by their "heuristic search" techniques (e.g., in the chesplaying programs, the techniques for selectively searching through a large number of possible chess moves), while the later programs were increasingly characterized by their techniques for the "representation of knowledge" (e.g., in the chess-playing programs, the representation of the higher-level concepts of chess strategy, in ways that might approximate the strategic concepts of human chess players). To see this shift in emphasis, compare the early articles collected in Computers and Thought (*E. Feigenbaum & J. Feldman*, eds., 1968) with the articles collected in Representation and Meaning: Experiments with Information Processing Systems (*H. Simon & L. Siklossy*, eds., 1972).

II. Some Fundamentals of Corporate Reorganization Law

There are several considerations involved in choosing an area of the law for a project like TAXMAN. The project must initially be a manageable one: it must be possible, with the existing techniques of semantic information processing, and with a reasonabge effort and a reasonable amount of time, to develop a computer model which performs at least a few interesting tasks, and which readily suggests extensions to at least a few more. But the project must not be too easily manageable, either. For in order to push the current paradigm to its limits, and to see the possible modifications, there must be a richness and a depth to the legal problem area that we would not expect to be able to capture in a trivial way. The key, then, is a carefully graduated complexity.

Certainly one of the best areas of the law in this respect is the taxation of corporate reorganizations. The problem here is to determine the tax consequences of certain rearrangements of corporate structure: mergers, acquisitions, divisions, divestitures, and the like. Some of the transactions here will be classified as a "sale," or as the receipt of "income," and thus taxable; others will be classified as part of a "reorganization," and thus tax-free. The problem is: which is which? There are aspects of this problem, as we shall see, which can be handled quite adequately within the current paradigm of semantic information processing. But there are also complexities in this area as troublesome as any in the entire legal system.

To convey briefly the basic concepts governing the taxation of corporate reorganizations, I will analyze in this section several variations on a simple hypothetical case[25]. Consider a New Jersey corporation which, for a variety of reasons, wishes to transfer its place of incorporation to Delaware. It chooses to do so by means of an "assets acquisition," as follows. The New Jersey corporation first sets up a new corporation in Delaware, then transfers all its assets to the Delaware corporation in exchange for newly issued Delaware stock. The result at this point is a hierarchical corporate structure, as shown in Figure 1(a), the New Jersey shareholders controlling a company which owns nothing but Delaware stock, and the New Jersey company operating the Delaware company as a wholly owned subsidiary. Most likely, however, the old New Jersey company would then be liquidated, and the Delaware stock distributed pro rata to the New Jersey shareholders in exchange for their New Jersey stock. The result at this point would be fully equivalent to a direct reincorporation of the New Jersey company in Delaware, as shown in Figure 1(b): the original shareholders would possess the same interest in the same

25 The hypothetical case that follows is based on United States v. Phellis, 257 U.S. 156 (1921), the earliest reorganization case. The Phellis case also appears in severa subsequent sections of this article: it provides a stock of examples in the discussion o semantic information processing, pp. 337–353 infra; it is used to demonstrate th actual performance of the TAXMAN system, pp. 353–358 infra; and it appears promi nently in a discussion of the early reorganization cases in Part Two. In using Phellis i this way as a device for the introduction to corporate reorganization law, I have draw heavily upon *W. Andrews*, Federal Income Taxation: Cases, Problems and Note 125–58, 168–69, 757–844 (1969); I have also drawn upon *B. Bittker & J. Eustice*, Feder Income Taxation of Corporations and Shareholders (3d ed. 1971).

Figure 1(a):

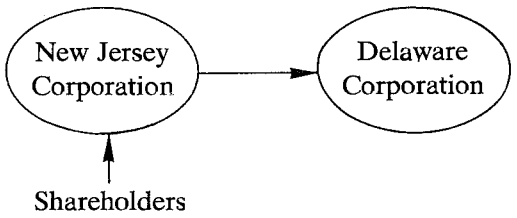


Figure 1(b):

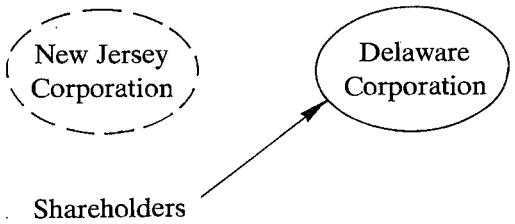
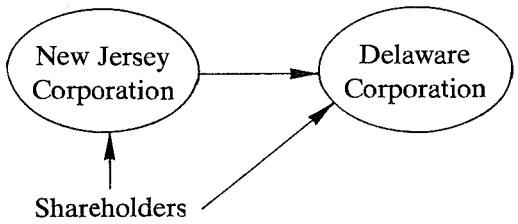


Figure 1(c):



operating assets as before, with the corporate form alone altered and transferred to a different state. Alternatively, and less likely, only a portion of the Delaware stock might be distributed to the New Jersey shareholders, while the New Jersey corporation itself remains in existence. If so, as shown in Figure 1(c), the original shareholders would then control the Delaware company partly by direct ownership of its stock, and partly by indirect ownership through New Jersey.

What would be the tax consequences of these transactions? Without special provisions in the Internal Revenue Code for a reorganization such as this, there would be a potential tax liability for both the shareholders and the corporation. The New Jersey shareholders would be potentially taxable on the receipt of the Delaware stock, since this stock in the hands of the New Jersey corporation would be like any other property, or cash, which the corporation might own and distribute. If the Delaware stock were completely distributed and the New Jersey corporation completely liquidated, as in Figure 1(b), the shareholders would be taxed at capital gains rates on the appreciation of their investment, i.e., on the difference between the market value of the Delaware shares received and the cost basis of

their original shares in New Jersey[26]. If only a portion of the Delaware stock were distributed in exchange for a portion of the shareholders' New Jersey stock, as in Figure 1(c), the transaction might qualify as a "stock redemption," subject also to capital gains treatment[27], or it might be found to be "essentially equivalent to a dividend"[28], and thus includible in the taxpayers' ordinary gross income[29]. In either event, but particularly in the latter, the tax consequences for the New Jersey shareholders could be quite burdensome. The New Jersey corporation could also find itself subject to a burdensome tax on the same series of transactions. The initial transfer of property to Delaware would be viewed as a "sale," and the New Jersey corporation would thus be taxed, at capital gains rates, on the value of the Delaware shares received less the cost basis of its transferred assets.

All of these results would seem a bit peculiar, however, or so it could be argued. In substance, the argument goes, there has not really been a liquidation, or a redemption, or a dividend, or a sale. The old corporation is identical to the new, in all respects but the formal state of incorporation. One way to see the paradox here is to focus on the distribution of Delaware stock to New Jersey shareholders, and compare this reorganization distribution to a comparable stock distribution outside of the reorganization context. If the New Jersey corporation had distributed its own stock to its own shareholders, there would be under most circumstances no tax liability to the recipients[30]: each shareholder would experience a certain percentage increase in the number of shares owned, but his proportional equity interest in the affairs of the corporation would remain the same, and the total effect would be closer to a stock split than a distribution of earnings and profits. On the other hand, if the New Jersey corporation had distributed to its shareholders the stock of a totally unrelated corporation, the distribution would clearly be taxable as an ordinary dividend. In our original case, then, a tax on the distribution of Delaware stock follows by analogy to this latter example. But at the time the distribution is made, the argument goes, the new Delaware company is nearly identical to the old New Jersey company and the new New Jersey company is nothing but a hollow corporate shell holding Delaware stock. Shouldn't the stock distribution thus be viewed as more nearly equivalent to a distribution by New Jersey of its own stock, as the equivalent of a non-taxable stock dividend? And if this argument is persuasive, shouldn't the liquidation be ignored also? Shouldn't the corporation be seen as continuing, unliquidated, in Delaware, while the liquidation of the old New Jersey shell is seen only as a formal reshuffling of stock certificates? Finally, if we agree to collapse the separate identities of the New Jersey and Delaware corporations, shouldn't we also view the sale of the New Jersey assets as a purely internal affair, as a shift of property from left hand to right hand, so to speak, and thus also immune from taxation?

When these arguments were presented to the Supreme Court in a series of cases

26 §§ 331(a) (1), 1001–02, 1221–22.
27 §§ 302(a)–(b), 317(b).
28 § 302(b) (1).
29 §§ 301(a), (c), 316.
30 § 305; *Eisner v. Macomber,* 252 U.S. 189 (1920). For a discussion of Macomber, and its relevance to the early reorganization cases, see Part Two of the present article.

in the 1920's[31], under a statute that contained no specific references to corporate reorganizations[32], the taxpayers, with one exception[33], lost. But even before these cases were decided, Congress had enacted in the Revenue Act of 1918 a rudimentary provision exempting from taxation the exchange of stock or securities "in connection with the reorganization, merger, or consolidation of a corporation . . ."[34]. Not surprisingly, the descendants of this simple declaration today are among the most complex of all the provisions of the Internal Revenue Code. The following account of the current statutory scheme is necessarily a simplification, but it contains the essential ingredients: the non-recognition provisions, the basis provisions, and the definitional provisions.

First, following the arguments made above in the New Jersey-Delaware hypothetical, the Internal Revenue Code today adopts a policy of recognizing taxable gain or loss to neither the corporation nor the shareholders in certain exchanges incident to a "reorganization." Under § 354, no gain or loss is recognized to a shareholder who exchanges "stock or securities in a corporation a party to a reorganization" solely for other stock or securities in a "corporation a party to the reorganization," provided the exchange is undertaken "in pursuance of the plan of reorganization"[35]. Likewise, under § 361(a), no gain or loss is recognized to "a corporation a party to a reorganization" which exchanges its property "solely for stock or securities in another corporation a party to the reorganization," provided again that the exchange is "in pursuance of the plan of reorganization"[36].

Second, when a gain or loss goes unrecognized in a reorganization exchange, the basis of the property exchanged is generally transferred over to the property received, thus making the unrecognized appreciation potentially taxable at a later date. Under § 358, therefore, the shareholders who receive stock or securities in the reorganization will carry their new certificates at the same basis as the old certificates they have given up, and the corporation which receives stock or securities in exchange for its property will carry the new certificates at the same basis as the transferred assets. Under § 362(b), the corporation which receives property in exchange for its own stock or securities will carry the new assets at the basis they possessed in the hands of the prior owner[37]. The net effect of the

31 United States v. Phellis, 257 U.S. 156 (1921); Rockefeller v. United States, 257 U.S. 176 (1921); Cullinan v. Walker, 262 U.S. 134 (1923); Weiss v. Stearn, 265 U.S. 242 (1924); Marr v. U.S., 268 U.S. 536 (1925). For a further discussion of these early reorganization cases, see Part Two of the present article.

32 Act of October 3, 1913, ch. 16, 38 Stat. 114, 166; Act of September 8, 1916, ch. 463, 39 Stat. 756.

33 Weiss v. Stearn, 265 U.S. 242 (1924).

34 Revenue Act of 1918, ch. 18, § 202(b), 40 Stat. 1057, 1060.

35 See also § 355, which applies to certain distributions and exchanges whether or not there has been a reorganization; and § 356, which governs the receipt of "other property or money" falling outside of either § 354 or § 355.

36 See also § 361(b), which governs the receipt of "other property or money" falling outside of § 361(a).

37 Note that § 362(b) establishes a "carry-over" basis rule, rather than the "substitute" basis rule of § 358. See generally, *B. Bittker & T. Eustice,* supra note 25, ¶ 14.33.

recognition and basis provisions taken together, then, is not to forego the tax entirely, but to defer it until the reorganization related property is eventually sold in a non-reorganization exchange.

Finally, since the recognition and basis provisions apply in full only to "reorganizations," that term is specified in elaborate detail in the definitional provisions of the Code, § 368(a). There are six types of reorganizations, known colloquially by the letter of the subsection in which they are defined. Focusing for the moment on the general pattern of the definitions and omitting some of the important detail, the six reorganizations are: (A) "a statutory merger or consolidation"[38]; (B) the acquisition by one corporation of "control" of another corporation, through the purchase of the acquired corporation's stock, and in exchange "solely for . . . voting stock" of the acquiring corporation[39]; (C) the acquisition by one corporation of "substantially all the properties" of another corporation, once again in exchange "solely for . . . voting stock" of the acquiring corporation[40]; (D) the transfer of assets by one corporation to another corporation which the transferor immediately thereafter "controls"[41]; (E) "a recapitalization"[42]; and (F) "a mere change in identity, form, or place of organization, however effected"[43]. The crucial concept of "control," which appears in the definitions of both a Type B and a Type D reorganization, is defined fairly mechanically as "the ownership of stock possessing at least 80 percent of the total combined voting power of all classes of stock entitled to vote and at least 80 percent of the total number of shares of all other classes of stock of the corporation"[44]. It should be noted also that the statute delineates the concept of "a party to a reorganization" in great detail[45], while the concept of a "plan of reorganization" is left conspicuously undefined, even though both terms play an important role in the main reorganization provisions.

In order to see the mechanics of these provisions more clearly, consider how our original New Jersey-Delaware hypothetical of Figure 1(b) would fit within the statutory scheme. In spirit and effect, the transaction there seems to be a Type F reorganization, a "change in . . . place of organization, however effected." But the case fits the general pattern of two other provisions as well. It could be viewed

38 § 368(a) (1) (A).

39 § 368(a) (1) (B).

40 § 368(a) (1) (C). See also § 368(a) (2) (B), which provides additional rules for the case in which the acquiring corporation exchanges "money or other property" in addition to voting stock.

41 § 368(a) (1) (D). There is an important addendum to this definition, however. In order to qualify under § 368(a) (1) (D), the plan of reorganization must include a distribution of the transferee's stock or securities which qualifies under § 354, § 355 or § 356. But according to § 354(b) (1), § 354 does not apply to a Type D reorganization unless the transferee acquires "substantially all of the assets" of the transferor, and the transferor in turn distributes all of its properties (including those received in the reorganization) in what would effectively be a complete liquidation. Thus any putative Type D reorganization involving less than substanially all the assets of the transferor, or less than a complete liquidation of the transferor, would have to satify the requirements of § 355.

42 § 368(a) (1) (E).

43 § 368(a) (1) (F).

44 § 368(c).

45 § 368(b).

as an acquisition by the Delaware corporation of "substantially all the properties" of the New Jersey corporation, i.e., a Type C reorganization. Or it could be viewed as a Type D reorganization, a transfer of assets by the New Jersey corporation to the Delaware corporation, which New Jersey then "controls" by virtue of its ownership of all the newly issued Delaware stock[46]. But in either case, the receipt of Delaware stock by New Jersey shareholders in exchange for their old New Jersey shares would be nontaxable under § 354, since New Jersey and Delaware are both parties to a reorganization[47]. The New Jersey corporation under § 361 would be subject to no tax liability on the transfer of its assets to Delaware; and the Delaware corporation, under § 1032, and quite apart from any of the reorganization provisions, would be subject to no tax liability on the transfer of its stock to New Jersey[48]. The New Jersey shareholders would then hold the Delaware stock, Figure 1(b), under § 358 at their old New Jersey basis, allocated proportionately if necessary; the Delaware stock that is temporarily retained by New Jersey, Figure 1(a), would have a basis under § 358 derived from the basis of the transferred New Jersey properties; and the transferred properties in Delaware's hands would retain the basis under § 362(b) that they had when they were previously owned by New Jersey. Thus, if any of these stocks or assets were subsequently sold, they would incur then the tax liability that was deferred at the time of the initial transaction.

For a somewhat different view of the reorganization provisions, particularly the definitional sections, consider the principal corporate purposes contemplated by the six approved reorganization patterns. Type A, Type B, and Type C reorganizations are intended primarily for the combination of two separate corporations into one, i.e., a straightforward statutory merger or consolidation, or the functional

46 According to § 368(a) (2) (A), a transaction described in both § 368(a) (1) (C) and § 368(a) (1) (D) will be treated as if it were described only in § 368(a) (1) (D). Also, according to Rev. Rul 57–276, 1957–1 C.B. 126, in the case of an overlap between a Type F reorganization and a Type C or a Type D reorganization, the transaction should be treated as a Type F reorganization, at least for purposes of § 381(b), where the distinction makes a difference. The full effect of these overlap provisions and rulings is uncertain. See *B. Bittker & J. Eustice,* supra note 25, ch. 14, at 58, 70–71. For present pedagogical purposes, however, it is sufficient simply to note the multiple application of these definitions; the more concepts the hypothetical case here illustrates, the more useful it is.

47 Notice that the hypothetical transaction depicted in Figure 1(b) has no difficulty fitting within the interacting provisions of § 354(b) (1) and § 368(a) (1) (D), supra note 41, since the New Jersey corporation there has transferred all of its assets to Delaware and then completely liquidated itself, as required. But the hypothetical of Figure 1(c) would not fare so well. If Figure 1(c) were analyzed as a Type D reorganization, § 354 would not apply because the New Jersey corporation has failed to distribute all of its properties; and § 355 would apply only if the New Jersey corporation has distributed enough of the Delaware stock to constitute "control" of Delaware within the meaning of § 368(c), and only if certain other requirements of § 355 (not mentioned in the hypothetical statement of facts) were satisfied as well. The transaction of Figure 1(c) could be analyzed as a Type C reorganization, but this would presumably be barred by § 368(a) (2) (A), supra note 46.

48 § 1032 provides that "no gain or loss shall be recognized to a corporation on the receipt of money or other property in exchange for stock (including treasury stock) of such corporation".

equivalent of a merger by means of a stock or assets acquisition. A Type D reorganization is intended primarily for the division of one corporation into two. Type E and Type F reorganizations are intended for simple adjustments in capital structure, place of incorporation, etc, within a single ongoing corporation. As we have seen, the New Jersey-Delaware hypothetical fits most closely this latter purpose: since all the New Jersey assets are transferred to a newly formed Delaware corporation with no assets of its own, there are no real changes in corporate structure before and after the transaction. But this same factual pattern can be modified slightly to fit either of the other two corporate purposes. Suppose New Jersey and Delaware were both pre-existing corporations with their own separate operating assets: then the transfer of all the New Jersey assets to Delaware would constitute a merger of New Jersey into Delaware, and the case would fit the pattern of a Type C reorganization, but probably no longer a Type D. For a different variation, suppose only a portion of the New Jersey assets are transferred to a newly formed Delaware: the transaction would then constitute a division of the single New Jersey corporation into two new corporations, and the case would qualify potentially as a Type D, but no longer a Type C.

The body of law just described has a highly formal character, of course. It appears to be governed by a large number of complex but very precise rules. One should not conclude, however, that reorganization law consists entirely of the mechanical manipulation of abstract statutory forms: this may be a necessary step[49], but it is not a sufficient one. Throughout the history of the Code, the courts have intervened repeatedly to deny a claimed reorganization status to transactions which, while fitting literally within the statutory rules, have seemed inconsistent with the underlying purposes and assumptions of these rules.

One early example can be seen in the doctrine of "continuity of interest" developed by the courts in a series of cases in the 1930's[50]. Prior to 1934, the predecessors of the definitional Type B and Type C clauses lacked the requirement that a reorganization acquisition be made in exchange "solely for voting stock" of the acquiring corporation, and it soon became apparent that without some such requirement the reorganization provisions would be applicable, literally, to transactions which were outright sales. Suppose the New Jersey shareholders in our original hypothetical decide to sell out their company entirely to the DuPont corporation, a large conglomerate in Delaware: they choose to do so by having New Jersey transfer all its assets to DuPont in exchange for DuPont's short-term notes, payable in two months; they then arrange the liquidation of New Jersey,

49 See, e. g., *Chirelstein,* Learned Hand's Contribution to the Law of Tax Avoidance, 77 *Yale L. J.,* 440, 440 (1968): "the ability to generate a multiplicity of formal alternatives, however sterile the exercise in any other context, is usually thought to be the true mark of a creative tax advisor", citing *B. Bittker,* Professional Responsibility and Federal Tax Practice 55 (1965).

50 Cortland Specialty Co. v. Commissioner, 60 F.2d 937 (2d Cir. 1932), cert. den., 288 U.S. 599 (1933); Pinellas Ice & Cold Storage Co. v. Commissioner, 287 U.S. 462 (1933); John A. Nelson Co. v. Helvering, 296 U.S. 374 (1935); Helvering v. Minnesota Tea Co., 296 U.S. 378 (1935); Helvering v. Watts, 296 U.S. 387 (1935); Le Tulle v. Scofield, 308 U.S. 415 (1940).

receiving the notes as a liquidating dividend. Under the original reorganization provisions, circa 1928, these transactions would have qualified as a tax-free Type C reorganization[51]: there would have been no tax to the New Jersey shareholders on receipt of the short-term "securities" in exchange for their stock; nor would there have been a tax to the New Jersey corporation on the initial sale of its assets. The shareholders would have paid a deferred tax when the short-term notes were paid, of course; but the corporation, under the 1928 statute, would have escaped the tax altogether[52]. We would thus have a situation quite the reverse of our earlier New Jersey-Delaware hypothetical, where a clear reorganization would be taxed, absent special provisions, as a sale: here a transaction which appears rather clearly to be a sale would be treated by the special provisions as a tax-free reorganization.

When a problem of this sort came to the Supreme Court in *Pinellas Ice & Cold Storage Co. v. Commissioner*[53], however, the Court denied the claimed reorganization status. The Court held that a transaction in which the original owners retained no continuing interest in their transferred assets was, indeed, equivalent to a sale and "to be within the exemption the seller must acquire an interest in the affairs of the purchasing company more definite than that incident to ownership of its short-term purchase-money notes"[54]. Alternatively, the Court held that the transferred assets were not exchanged "solely for stock or securities in another corporation a party to the reorganization," under what is now § 361(a), since the notes "were not securities within the intendment of the act and were properly regarded as the equivalent of cash"[55]. Today, of course, the transferor's continuity of interest in a Type B or Type C reorganization is assured by the "solely for voting stock" requirement, but the "stock or securities" phrase is still defined only by judicial decision, and the "continuity of interest" doctrine itself remains viable in other reorganization contexts[56]. We will consider the reasoning of the *Pinellas* court in greater detail in a later section of this article[57]. For our present purposes, it

51 See, e. g., Revenue Act of 1928, ch. 852, § 112 (i) (1) (A), 45 Stat. 791, 818.

52 The situation would be different today: if this were a reorganization under the current statute, the DuPont corporation would carry the transferred assets under § 362(b) at the same basis they possessed in the hands of the New Jersey corporation, and any appreciation in the value of these assets would be potentially taxable at a later date. But the parallel provision in the 1928 statute would not have covered this hypothetical case unless the New Jersey shareholders managed to obtain "control" of the DuPont corporation immediately after the transfer; otherwise, the DuPont corporation would have received the assets with a stepped-up fairmarket-value basis, and would thus have escaped entirely the potential tax. Revenue Act of 1928, ch. 852, § 113 (a) (7), 45 Stat. 791, 819. For a further discussion of the significance of these basis provisions in this context, see *W. Andrews,* supra note 25, at 789.

53 287 U.S. 462 (1933). The same position had been taken a year earlier by the Second Circuit in Cortland Specialty Co. v. Commissioner, 60 F.2d 937 (2d Cir. 1932), cert. den., 288 U.S. 599 (1933).

54 287 U.S. 462, 470 (1933).

55 Id. at 468.

56 For example, it remains important for Type A and Type E reorganizations, where there is no statutory codification of a "continuity of interest" rule. See generally, *W. Andrews,* supra note 25, at 810–16; *B. Bittker & J. Eustice,* supra note 25, ch. 14, at 25, 27–28, 69.

57 See Part Two of this article.

is enough to note the decision as a conspicuous exception to the formal application of statutory rules described so far, and thus as an important phenomenon which our theories and models of reorganization law must ultimately attempt to explain.

For a more exotic example, consider the problems presented in the celebrated case of *Gregory v. Helvering*[58]. Suppose our New Jersey corporation owns a block of General Motors stock which has appreciated substantially in value; and suppose the New Jersey shareholders wish to sell the stock, and realize a substantial profit, but a direct stock dividend, as we have seen, would be taxable at ordinary income rates. Assume that New Jersey thus embarks on the following plan of "reorganization." A new corporation is established in Delaware, and the General Motors stock is transferred to it in exchange for a controlling amount of newly issued Delaware stock, which is then distributed to the New Jersey shareholders. Under the 1928 statute, this transaction would have qualified as a Type D reorganization[59], and neither the transfer of the General Motors stock nor the distribution of the Delaware stock would have been taxable. The Delaware corporation would then have been liquidated, and the General Motors stock distributed to its shareholders: as a liquidating dividend, of course, this distribution would normally have been taxable as a capital gain.

But should we permit the reorganization forms to be used in this way as a device for the avoidance of taxes? The Second Circuit and the Supreme Court denied the capital gains treatment in such a situation in *Gregory*, even while admitting that "the facts answer the dictionary definitions of each term used in the statutory definition"[60], and that a taxpayer "may so arrange his affairs that his taxes shall be as low as possible"[61]. According to the Supreme Court:

> The whole undertaking, though conducted according to the terms of subdivision (B) was in fact an elaborate and devious form of conveyance masquerading as a corporate reorganization, and nothing else. The rule which excludes from consideration the motive of tax avoidance is not pertinent to the situation, because the transaction upon its face lies outside the plain intent of the statute. To hold otherwise would be to exalt artifice above reality and to deprive the statutory provision in question of all serious purpose[62].

58 293 U.S. 465 (1935), aff'g. 69 F.2d 809 (2d Cir. 1934).

59 Revenue Act of 1928, ch. 852, § 112 (i) (1) (B), 45 Stat. 791, 818. Under the current statute, though, the transaction would not satisfy the final clause of § 368 (a) (1) (D): § 354 would not apply because only a portion of the New Jersey assets are transferred, and because the New Jersey corporation is never liquidated; § 355 would almost certainly not apply because of the "acitve business" requirement of § 355 (b) and the "device" restriction of § 355 (a) (1) (B). See notes 41 and 47, supra. In fact, the current provisions of § 368 (a) (1) (D), § 354 and § 355 are intended precisely to prevent a tax-free result in a case like *Gregory*.

60 69 F.2d 809, 810 (2d Cir. 1934).

61 Id.

62 293 U.S. 465, 470 (1935). The principle developed in Gregory is known generally as the "business purpose" doctrine. It is restated in similar language in the Regulations under § 368: "A scheme, which involves an abrupt departure from normal reorganization procedure in connection with a transaction on which the imposition of tax is imminent, such as a mere device that puts on the form of a corporate reorganization as a disguise for concealing its real character, and the object and accomplishment of which is the

Again, while deferring a detailed analysis of this decision to a later section[63], it should be clear at this point that the reorganization law created and applied here is something more than the mechanical manipulation of statutory rules. In fact, the case shows something quite the contrary: a taxpayer attempting to hide behind the formal manipulation of the statute, while a court cuts across the formalities and constructs a new set of concepts to deny the apparent statutory conclusion.

Finally, for a somewhat similar example, consider the rather complex manipulations of *Helvering v. Elkhorn Coal Co*[64]. In this case, because of a problem in satisfying the "substantially all properties" clause of a Type C reorganization, the transferor corporation strung together in sequence a Type D, a Type C and a Type B reorganization to get the result it wanted. Paraphrasing the facts again in terms of our New Jersey-Delaware hypothetical, we assume that New Jersey wishes to transfer some, but not all, of its assets to the DuPont corporation in a tax-free exchange. First, New Jersey transfers the assets it wishes to retain to a newly created Delaware corporation in a Type D reorganization, distributing the Delaware stock; second, New Jersey transfers all its remaining assets to DuPont in exchange for voting stock, claiming a Type C reorganization; finally, Delaware acquires all the outstanding New Jersey stock from the New Jersey shareholders in exchange for its own voting stock, a Type B reorganization. When Delaware subsequently liquidates New Jersey, the transaction is complete: the Delaware corporation has now stepped completely into the former position of the New Jersey corporation, retaining only the wanted assets plus the stock from DuPont. Once again, under the 1928 statute, the definitions of a Type D, a Type C and a Type B reorganization would have been precisely satisfied[65]. But would the courts permit these formally tax-free steps to be combined into a single tax-free reorganization? The Fourth Circuit in *Elkhorn* would not. The transfer of assets to the newly created corporation served no purpose, the court said, except to strip these assets away temporarily in preparation for the contemplated Type C reorganization; and since this first step was "a mere shifting of charters"[66], the subsequent step had only the appearance of being a transfer of "substantially all the properties" of the transferor; in reality, only a portion of the assets of the old company were transferred, and the transaction would therefore be taxable as a sale; this result, the court said, is "not made a different result because reached by following a devious path"[67].

---

consummation of a preconceived plan having no business or corporate purpose, is not a plan of reorganiziation", Treas. Reg. § 1.368–1 (c) (1955).

63 See Part Two of this article.

64 95 F.2d 732 (4th Cir. 1938), cert. den., 305 U.S. 605 (1938).

65 Revenue Act of 1928, ch. 852, § 112 (i) (1) (A)–(B), 45 Stat. 791. In this last hypothetical case, unlike the earlier ones, the current statutory definitions of a Type D, a Type C, and a Type B reorganiziation might be satisfied as well: in particular, the requirements of § 368 (a) (I) (D) and § 355 could be satisfied if both the assets transferred and the assets retained constituted an "active trade or business". For a discussion of several variations on this theme, see *B. Bittker & J. Eustice,* supra note 25, ¶ 14.52.

66 95 F.2d 732, 734 (4th Cir. 1938).

67 Id., at 738 (on rehearing), quoting Minn. Tea Co. v. Helvering, 302 U.S. 609, 613 (1937). The principle developed here is generally referred to as the "step transaction" doctrine.

Although the preceding analysis of the taxation of corporate reorganizations has been somewhat cursory, it should now be clear why this area of the law is ideally suited for a project like TAXMAN. There is a manageable level of complexity here, to start with. The basic factual situations that arise can be described in relatively simple terms – – corporations and shareholders, transferring stocks and assets, of various kinds, in various ways. Given these factual situations, most of the legal concepts in the area have a clear-cut, well-defined profile. But superimposed on this rather manageable foundation is another system of concepts as unruly as any that can be found elsewhere in the law. All of the paradoxes of the legal process and the classical dilemmas of legal reasoning appear here: there are familiar contrasts between "form" and "substance;" contrasts between statutory "rules" and judicially created "principles"[68]; contrasts between "legal formality" and "substantive rationality" and the possible middle ground between the two[69]. And because the foundation is itself so clear, the difficulties and ambiguities appear here in a particularly sharp light.

It is perhaps no accident that two of the more quotable passages on the problems of legal interpretation have arisen in corporate tax cases. Thus the famous metaphor of Holmes – – "a word is not a crystal, transparent and unchanged, it is the skin of a living thought" – – appeared first in *Towne v. Eisner*[70], the earliest stock dividend case. And the simile of Learned Hand – – "the meaning of a sentence may be more than that of the separate words, as a melody is more than the notes" – – appeared first in *Helvering v. Gregory*[71]. In a sense, the ultimate task for the TAXMAN project is to translate these familiar metaphors and similes into something more precise and concrete. But I should emphasize here that this is indeed an ultimate task – – the kind of issue that might properly be taken up in Part Two of this article. The current version of TAXMAN comes nowhere near a formalization of these ideas, nor does it even try. To see more clearly what the current program can accomplish, and what it cannot, we need to look more closely at the current paradigm of semantic information processing – – to which we now turn.

III. Some Fundamentals of Semantic Information Processing

In the Introduction to this article, I characterized the TAXMAN system as an attempt to represent the basic concepts of corporate reorganization law in a functioning computer program, and in the following pages I will describe briefly – – for the benefit of the nonspecialist – – how this is done. The problem can be simply stated. We wish to develop models of conceptual structures which are at least rough approximations of the conceptual structures developed by the human mind. And we wish to show how these conceptual structures can be represented and stored in computer memory; how they can be searched for and retrieved from

68 See, e. g., *Dworkin,* The Model of Rules, 35 U. Chi. L. Rev. 14, 22–29 (1967).
69 See, e. g., *Kennedy,* Legal Formality, 2 J. Legal Studies 351, 354–60, 391–98 (1973).
70 245 U.S. 418, 425 (1917).
71 69 F.2d 809, 810 (1934).

computer memory; and how they can be manipulated to simulate some of the more rudimentary patterns of human reasoning and inference.

In the past several years, much of the work on this problem has coalesced into the development of a family of programming languages designed especially for these purposes, and thus one of the best ways to analyze the current ideas in this branch of computer science is to study one of these programming languages in detail. In the exposition that follows, I will use a simplified version of the programming language in which TAXMAN is actually written[72], which is itself one of the earliest members of the family[73]. Despite this simplification, the version of the language I will use exhibits most of the salient features – – and most of the outstanding problems – – of the current paradigm of semantic information processing[74].

First of all, what sorts of concepts do we want to represent? At a bare minimum, it should be clear from the discussion in the preceding section that we need to express certain fundamental propositions about corporations: that a particular corporation exists; that it has issued certain classes of stock; that certain individuals or other corporations own various shares of that stock; that the ownership of these shares of stock has been transferred from time to time. In some sense, these sorts of propositions seem to be the building blocks upon which most of the rest of

---

72 TAXMAN is written primarily in a language called micro-*Planner*, which is a subset of a (not fully implemented) language called *Planner*. See, *G. Sussman, T. Winograd & E. Charniak,* Micro-*Planner* Reference Manual, AI Memo No. 203 (Mass. Inst. of Technology, 1970); *B. Baumgart,* Micro-*Planner* Alternate Reference Manual, Stanford AI Lab Operating Note No. 67 (Stanford Univ. 1972); *C. Hewitt,* Description and Theoretical Analysis (Using Schemata) of *Planner:* A Language for Proving Theorems and Manipulating Models in a Robot, AI Memo No. 251 (Ph.D. Dissertation, Mass. Inst. of Technology 1972). Micro-*Planner* itself is a high-level language written in a lower-level language called *Lisp,* which has been one of the principal tools for artificial intelligence research for over a decade. See, *J. McCarthy, P. Abrahams, D. Edwards, T. Hart & M. Levin, Lisp* 1.5 Programmer's Manual (1962); *C. Weissman, Lisp* 1.5 Primer (1967). Portions of TAXMAN are also written directly in *Lisp.*

73 These programming languages are constantly evolving and proliferating, as new ideas are generated and as new deficiencies are noted in the earlier versions. For example, the dissatisfaction of one group of users with *Planner* has led to the development of a variant known as *Conniver.* See, *Sussman & McDermott,* from *Planner* to *Conniver* – a genetic approach, 1972 Proceedings of the Fall Joint Computer Conf. 1171 (Am. Fed'n of Information Processing Societies, Montvale, N. J.); *D. McDermott & G. Sussman,* The *Conniver* Reference Manual, AI Memo No. 259 (Mass. Inst. of Technology 1972). See also, *J. Rulifson, J. Derkson & R. Waldinger,* QA4: A Procedural Calculus for Intuitive Reasoning, AI Center Technical Note 73 (Stanford Research Institute, 1972); *R. Reboh & E. Sacerdoti,* A Preliminary *QLISP* Manual, AI Center Technical Note 81 (Stanford Research Institute, 1973). For a comparative survey of these and related languages, see Bobrow & Raphael, New Programming Languages for Artificial Intelligence Research, 6 Computing Surveys 153 (1974).

74 Significantly, the work on natural language understanding by *T. Winograd,* supra note 20, uses the micro-*Planner* programming language as a central component, id., at 23–25, 108–17. For a further discussion of the role of a programming language such as micro-*Planner* in a full system for natural language understanding, see pp. 364–366 infra. The work in cognitive psychology cited supra note 21 has also been influenced substantially by both *Planner* and *Lisp.* See, e. g., *J. R. Anderson & G. Bower,* supra note 21, at 67–69, 125–28.

reorganization law is constructed: for reorganization purposes at least, they function as relatively concrete, relatively primitive facts. It should also be clear from the preceding discussion that we need to express a range of concepts which function within reorganization law at a much higher level of abstraction: "stock distribution," for example; or the concept of "control;" or the conclusory legal concept of a "tax-free reorganization" itself. In fact, a major portion of the task here would seem to be the representation of these various levels of conceptual abstraction, and the connection of one such level to another.

I will begin with the simplest representational problems. If we want to say something about a corporation named "New Jersey," and therefore want to express initially the proposition that "New Jersey is a Corporation," we can do so by storing in computer memory the simple *list* of two *words*: *(Corporation New-Jersey).* This list is stored exactly as written, both words spelled out in full, and it is indexed in such a way that by specifying either of the words *Corporation* or *New-Jersey* we can find the location of the list in memory and retrieve it again. The conventions for interpreting such a list are similar to those of formal symbolic logic: the first word is a *predicate*; the remaining words are *objects* to which the predicate is applied. If we also want to express the proposition that "Delaware is a Corporation," we could obviously do so by storing a second list *(Corporation Delaware).* Here, *Corporation* would be the same one-place predicate as before, with the same interpretation, but applied now to an object named *Delaware.* The indexing system could now be used to retrieve either *(Corporation New-Jersey),* by specifying *New-Jersey,* or *(Corporation Delaware),* by specifying *Delaware,* or both *(Corporation New-Jersey)* and *(Corporation Delaware)* together, by specifying the word *Corporation*[75].

As we move to representational problems of slightly greater complexity, of course, it becomes necessary to establish a number of conventions for translating ordinary English concepts into computer data structures. To a large extent, these conventions will be quite arbitrary. But in many instances, where there are several alternative representations to choose from, our choice of conventions can be guided primarily by pragmatic criteria, and the computer data structures can thus be closely tailored to the particular purposes for which they will be used.

For an example, consider the problems of representing the simple fact that New Jersey, a corporation, has issued stock. It would be possible to store this fact as a single list: *(Issue New-Jersey Stock),* but this choice would then encounter difficulties if it became necessary to say anything further about the particular stock

75 In this paragraph I have implicitly introduced some of the syntax of the *Lisp* programming language: note the manner in which a list like *(Corporation New-Jersey)* is composed by stringing together a sequence of words (or atoms) like *Corporation* and *New-Jersey,* and enclosing them inside a pair of parentheses. A list can also be composed by stringing together and parenthesizing a sequence of sublists, like *(List (Corporation New-Jersey) (Corporation Delaware)),* and this nesting of lists and sublists can continue to an arbitrary depth. For a discussion of these syntactic conventions, see, *C. Weissman,* supra note 72, at 5–24. For a description of the way in which micro-*Planner* uses these *Lisp* facilities to store and index a proposition like *(Corporation New-Jersey),* see *B. Baumgart,* supra note 72, at 14–16.

that New Jersey had issued – that it was common stock, that it was worth so many dollars per share, etc. The difficulty arises because *Stock* is not a particular object itself, but is instead a predicate characterizing a large class of particular objects. The list *(Issue New-Jersey Stock)* therefore provides us with no symbol in computer memory upon which to attach an additional proposition about the particular object in question. The solution to this problem is simple, however. We will have the computer program generate an arbitrary symbol to stand for the object, *S1*, say, and then store not one, but two lists: *(Issue New-Jersey S1)* and *(Stock S1).* Later, if we wish to say that the stock issued by New Jersey is worth so many dollars per share, we need only add to computer memory the fact that *S1* is worth so many dollars per share, relying on the previously stored information to tell us that *S1* is a stock and that *S1* was issued by New Jersey.

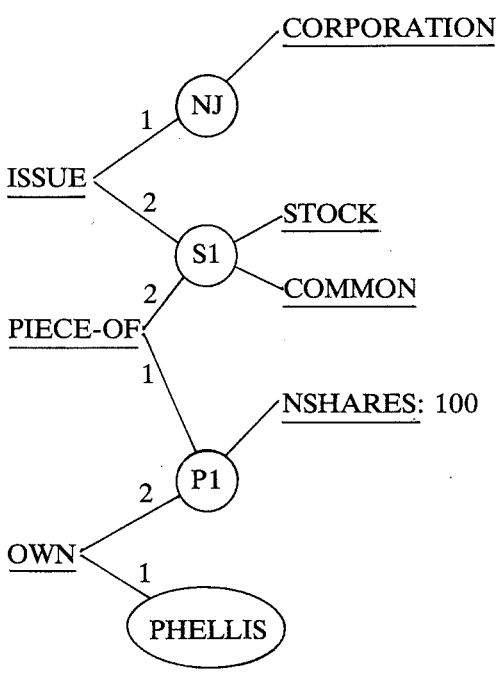
As it turns out, this kind of problem and its solution occur frequently in the TAXMAN system. One of the basic conventions in TAXMAN, therefore, is to *nominalize* the concepts of corporate reorganization law as much as possible: whenever linguistic proprieties permit us to treat a conceptual structure as an object, we will do so, setting up an internal symbol like *S1* to stand for the postulated entity. In making this choice, of course, we are saying nothing at all about whether an "object" which is described as a "stock" does or does not "exist", in some metaphysical sense; nor, in using a list like *(Corporation New-Jersey),* are we saying anything about whether an "object" which is described as a "corporation" does or does not "exist". In both cases, the object-oriented representation is chosen simply because, in these contexts at least, it is computationally convenient.

For another example of the conventions for representing concepts in TAXMAN, consider how to represent a stockholder's ownership interest in some particular class of stock. One possibility is to translate a proposition like "Phellis owns New Jersey stock" into the list: *(Own Phellis S1),* where *S1* is given the same implicit meaning as before. However, for present purposes at least, we will need a bit more flexibility in the ownership concept than this single list can provide: after all, a number of other stockholders also "own" *S1*; Phellis himself only "owns" a certain number of shares of *S1*; and any stockholder at any time can transfer all or part of his "ownership" interest in *S1* to someone else. To represent these possibilities, we adopt the convention that Phellis be said not to own *S1* directly, but rather to own some other entity called *P1* which is itself a "share-of" or a "piece-of" *S1.* Thus the proposition here would be stored as two lists: *(Own Phellis P1)* and *(Piece-of P1 S1).* With this convention, it is possible to talk separately about what Phellis owns and what other stockholders own; to attach to Phellis' interest another description telling how many shares it represents; and to modify these descriptions at any time to represent a transfer of Phellis' interest to someone else. Once again, the basic convention of pragmatic nominalization has suggested a way to expand and fragment a complex concept to fit a particular need, generating new symbolic entities wherever necessary. Obviously, too, should the present formalism later prove inadequate in some other context, it could be expanded once more. In principle, at least, this procedure could be repeated again to whatever degree of complexity and density seems useful.

By now I have developed enough semantic conventions in this discussion to represent a moderately complicated cluster of facts. Consider the proposition that "Phellis owns 100 shares of common stock issued by New Jersey". In the present formalism this factual cluster could be stored as:

(CORPORATION NEW-JERSEY)
(ISSUE NEW-JERSEY S1)
(STOCK S1)
(COMMON S1)
(PIECE-OF P1 S1)
(NSHARES P1 100)
(OWN PHELLIS P1)

It may be helpful, however, to exhibit an alternative means of visualizing this data structure, transforming it from a series of disconnected lists into an equivalent interlocking *network*. Notice that the objects in the structure –*New-Jersey, S1, P1,* and *Phellis* – are linked together in paris by the predicates: *Phellis* and *P1,* for example, are linked together by their appearance in the list with *Own; P1* and *S1* are linked together by their appearance with *Piece-of; New-Jersey* and *S1* are linked together by *Issue.* The existence of these linkages suggests that we draw a diagram of the data structure as follows:



Here, the objects are drawn as circular nodes, and the predicates are underlined and linked appropriately to one, two, or potentially more, of the objects. The diagram thus exhibits clearly the corporate structure we are trying to describe: in fact, it is quite similar in some respects to the diagrams commonly drawn by tax

lawyers to sort out the facts of a complex reorganization case[76]. If we prefer, of course, we can consider these network diagrams to be the primary mode of representation here, instead of the internal computer data structures initially introduced. Obviously, the two formalisms are equivalent, or isomorphic, the one being directly transcribable into the other.

Now given these rudimentary conceptual structures, in whatever formalism seems most natural, the next step is to define some of the fundamental operations that the TAXMAN system ought to perform on them. I have already mentioned the simplest of these operations: at a minimum, we need to *store* a given list in memory, *retrieve* it from memory, and then, under some circumstances, *delete* it from memory again. The storage and deletion operations will be accomplished by the commands *Assert* and *Erase*[78]. The command *(Assert (Corporation New-Jersey))*, for example, will call up a special program that stores and indexes the list *(Corporation New-Jersey)*, the same storage and indexing operation discussed implicitly before, but now given an explicit definition. Note the intended semantic interpretation of this command. To "assert" the proposition that "New Jersey is a Corporation" means to store the corresponding list in some identifiable location in memory, and thus to make it readily available for subsequent retrieval – that is, to render it, in some operational sense, "true". Similarly, the command *(Erase (Corporation New-Jersey))* will remove the list *(Corporation New-Jersey)* from memory, thus making it no longer available for retrieval – or, in some operational sense, "no longer true". In this way, the truth or falsity of a proposition is represented simply by its presence or absence in a particular portion of memory, a crude device, certainly, but for many purposes an effective one[79].

To retrieve a proposition from memory, a somewhat analogous *Goal* command will be used, but this *Goal* operation requires a slightly more detailed explanation than did *Assert* or *Erase.* It is useful to distinguish several types of retrieval, depending on what information is to be retrieved and what information is available already to guide the retrieval process. Thus, if we are considering the proposition *(Corporation New-Jersey)* and want to know whether this proposition has already been asserted in the data base, the command *(Goal (Corporation New-Jersey))* will provide the answer. The *Goal* program will search the data base for the list *(Corporation New-Jersey),* and, if successful, return *T*, otherwise *Nil.* Intuitively,

76 For example, compare this network diagram to the diagrams of corporate structure in Figure 1, supra.

78 These *Assert* and *Erase* commands (as well as some of the other operations in the discussion which follows, e. g., *Goal, Prog* and *Theorem)* are basic components of the micro-*Planner* programming language. See *G. Sussman, T. Winograd & E. Charniak,* supra note 72; *B. Baumgart,* supra note 72.

79 A more sophisticated approach might explicitly mark a proposition in memory as true or false. The absence of a proposition from memory might then be interpreted as an expression of ignorance about its truth or falsity, thus providing three truth-values: true, false and indeterminate. The *Conniver* language takes essentially this approach. See *D. McDermott & G. Sussman,* supra note 73. In a sense, the *Planner* convention followed here collapses falsity and indeterminance into a single truth-value, and the system can be thought of as maintaining a "model" of everything "known to be true.".

the *Goal* command tries to establish the "goal" of showing that the proposition "New Jersey is a Corporation" is "true" in the given data base. This is, indeed, the simplest form of retrieval: the straightforward verification that a particular assertion is present or absent – true o false – in memory.

There are other retrieval possibilities, however. What if we wanted to find out whether there was an assertion in the data base about any corporation at all, whatever its name might be? For this purpose we could use the command *(Goal (Corporation?))*, where the name of a specific corporation has been replaced by a question mark. The program would then search the data base for any list whose first element, the predicate, matched the word *Corporation,* whatever its second element might be. If it found the list *(Corporation New-Jersey),* for example, the match would be successful, and the program would return this list as the result of its search; otherwise, if it found no list that matched, it would return *Nil.* Intuitively, the command is asking: Is there any object "X" about which it has been asserted that "X is a corporation"? A slight variation of this request would be the command *(Goal (Corporation ?X)).* Here, the intuitive idea of matching a variable to an object is taken more seriously. The symbol *?X* refers literally to a variable named *"X"* which can be assigned several different values as the computation proceeds. The command *(Goal (Corporation ?X))* would search the data base for a list whose first element matched *Corporation,* as before, but, if the match succeeded, the variable *X* would then be assigned as its value the name of the object it matched. For example, if *(Corporation ?X)* is matched to *(Corporation New-Jersey), X* would be assigned the name *New-Jersey,* and any subsequent references to *X* would be interpreted, by substitution, to be references to *New-Jersey.* As a final variation of these retrieval programs, we need some mechanism to handle the possibility that two or more propositions about a corporation may appear simultaneously in the data base. The original *Goal* commands were designed to search for only one matching list, and thus if the data base contained both *(Corporation New-Jersey)* and *(Corporation Delaware),* the program would return whichever one it happened upon first. As an alternative, we need a command like *(Find all (Corporation ?))* which would return a list of all possible matches, in this case the list *((Corporation New-Jersey) (Corporation Delaware))*[80].

At this point, it may be useful to pause and take stock of the discussion so far. Paring away most of the details, the ideas I have introduced here have fallen into several basic categories. First, I have defined a *data base* in which we can formally represent some of the concepts necessary to describe the facts of a corporate reorganization case. Second, I have definied a set of *operations* on this data base, enabling us to add new data structures to it, modify old ones in it, and retrieve information generally about the state of affairs it represents. Also, in the discussion of the various *Goal* commands, I have introduced implicitly the important idea of *pattern-matching:* recall that a major feature of these retrieval programs was the

80 These simple examples do not exhaust all the retrieval possibilities, of course, but for present purposes they will be sufficient. For a survey of some additional conventions for assigning variables, and some techniques for matching and retrieving more complex data structures, see Bobrow and Raphael, supra note 73, at 164–66.

ability to match a partially specified pattern in the *Goal* statement against the fully specified patterns stored in the date base. So far, however, the repertoire of operations described here has been severely limited; the program has dealt only with single lists and single propositions, storing and retrieving them one at a time. As a minimal extension of these operations, it might be useful to store and retrieve a somewhat larger chunk of data structure – a network of related propositions, for example. Can we define a set of operations to do this? In terms of the pattern-matching idea just introduced, can we conceive of a program that matches patterns of greater structural complexity than the single lists which have so far appeared in the *Goal* commands?

For a first step in this suggested direction, consider the problem of making a simple logical deduction from a set of propositions stored in the data base. Suppose the data base contains a representation of the network depicted in Figure 2, and suppose we wish to establish the truth of the proposition "Phellis is a stockholder of New-Jersey". Since we previously decided to decompose the concept of stock ownership into several component parts, there is no single proposition in the network that expresses directly the relationship between a corporation and its stockholders. But obviously the stockholder relationship is expressed indirectly by the sequence of propositions involving *Issue, Stock, Piece-of* and *Own.* What is needed, therefore, is a command to search for all four of these related propositions in succession. Since this command will have some of the characteristics of an ordinary sequential computer program, it will be given the name *Prog* and required to have the following form: (*Prog* ⟨list of variables⟩ ⟨sequence of operations⟩)[81]. But the *Prog* command will also have some important differences from the ordinary computer program, as will soon become apparent.

Let us examine in some detail the use of the *Prog* command to establish the truth of the proposition "Phellis is a stockholder of New Jersey". The problem can be expressed as follows:

```
(PROG (S P)
(GOAL (ISSUE NEW-JERSEY ?S))
(GOAL (STOCK ?S))
(GOAL (PIECE-OF ?P ?S))
(GOAL (OWN PHELLIS ?P))
```

In this expression, the list *(S P)* is interpreted as a list of variables to be used in the computation, in exactly the same way that *X* was used as a variable in the example of a *Goal* command discussed earlier. The list of four *Goal* commands is then interpreted as a list of operations to be executed, one after another, so long at least

81 The angled brackets are used here to indicate in skeletal form the type of information which the command requires; this skeletal form should be compared with the full example given in the subsequent text. The term "ordinary computer program" is used here to refer to the types of programming languages that most people are most familiar with: *Fortran, Basic, Algol, PL/1,* etc. A subroutine in these languages contains a sequence of operations to be executed, and a list of variables to be used while the subroutine is running. *Prog* is an "ordinary computer program" in this respect.

as each *Goal* command is successful in its search of the given network. Thus, the first thing the program does here is to execute the command *(Goal (issue New-Jersey ?S))*, where *S* is a variable: obviously, this command will succeed in the given network, and the variable *S* will be matched to the object *S1*. Next, the command *(Goal (Stock ?S))* is executed: here, since *S* has already been assigned the value *S1*, this second command amounts to a search for the constant list *(Stock S1)*, which also succeeds. The third *Goal* command then becomes a search for a list to match *(Piece-of ?P S1)*, where *P* is still a variable, but where the original variable *S* has been replaced by the constant object *S1*: the search succeeds, of course, and the variable *P* is matched to the object *P1*. Finally, the fourth *Goal* command becomes a simple search for the list *(Own Phellis P1)*, which is also easily satisfied. The net result, therefore, is the successful execution of the original *Prog* command, and it is easy to see that this is equivalent to a successful search for the stockholder relationship, as desired. Intuitively, the program has found two objects, *S* and *P*, such that: "New Jersey has issued *S*, *S* is a stock, *P* is a piece-of *S*, and Phellis owns *P*." In other words: "Phellis is a stockholder of New-Jersey"[82].

The preceding example was too simple, of course, since the network yielded its answers much too readily: each search request was satisfied, immediately and uniquely. But suppose the network were more complex, showing the New Jersey Corporation with several stocks and bonds outstanding, and showing each of these stocks and bonds with several distinct stockholders and bondholders. How would the *Prog* command then determine that "Phellis is a stockholder of New Jersey?" The answer lies in the use of the variables *S* and *P* to store the matched objects only temporarily, and the ability of the *Prog* command to *backtrack* and try again whenever an initially successful match turns out later in the computation to be incorrect. For example, in the more complex network just described, it is possible that the first *Goal* command would be satisfied by retrieving the list *(Issue New-Jersey S7)* and thus assigning to the variable *S* the value *S7*. But it may then turn out that this object named *S7* is not a stock at all, but rather a bond, and so the subsequent search for *(Stock S7)* would fail. If so, the program would back up to the first *Goal* command and search there for another match. If it then found the list *(Issue New-Jersey S1)* on this second turn, it would retrieve it, re-assign *S* to the new object SI, and proceed forward again to search for *(Stock S1)*. Continuing in this manner, the program would eventually winnow out all the other incorrect matches as well – the wrong *Stocks*, the wrong *Pieces*, etc. – until it arrived at a choice of *S* and *P* that would finally enable it to execute successfully the fourth command: *(Goal (Own Phellis ?P))*. The net result, again, would be the successful execution of the original *Prog* command.

To sum up: No matter how complex the network might be, and no matter how many spurious stocks and bonds it might contain, the *Prog* command with its

82 Since all four of these *Goal* statements must be satisfied in order to satisfy the *Prog* command, they can be thought of as being implicitly connected by a logical *And*. Micro-*Planner* also provides for the explicit use of the other logical connectives: *Or*, *Not* and *If . . . Then*. See *G. Sussman, T. Winograd & E. Charniak*, supra note 72; *B. Baumgart*, supra note 72. It is thus possible to represent in a *Prog* command an arbitrarily complex proposition.

backtracking capability would still be able to determine whether or not "Phellis is a stockholder of New Jersey." It may be helpful to think of this process as an essentially "parallel" search, although constrained necessarily to operate in a "sequential" mode. What we really want the program to do is to match the two variables and satisfy the four commands simultaneously, but we accomplish this result by an interative, exhaustive, trial-and-error procedure[83]. It may also be helpful to think of this process as a somewhat more complex version of the pattern-matching operation discussed previously: here, rather than matching the data base network with a single list, we are matching it with another small chunk of network, a pattern consisting itself of four interlocking lists. But the basic idea of pattern-matching is the same in both cases, and this pattern-matching idea will later be extended to handle even greater complexities, and an even broader range of applications[84].

One important application of the *Prog* command at this point lies in the construction of *definite descriptions*. Suppose, for example, that we wanted to assert: "The value of the New Jersey stock owned by Phellis is $ 5000". If we knew that Phellis' ownership interest in this stock was represented by the symbol *P1*, we could store this assertion with the simple command *(Assert (Value P1 5000))*. However, in a network of any complexity, we would not be expected to know the names of all such internally generated symbols. Indeed, the very purpose of these internal symbols is to nominalize certain concepts which are not given proper names in ordinary discourse. But note that the English sentence singles out the object here by means of its definite description – "the New Jersey stock owned by Phellis" – and the *Prog* example analyzed above accomplishes essentially the same result when it locates an object *P* such that: "New Jersey has issued *S*, *S* is a stock, *P* is

83 Although this backtracking capability is an important programming idea (and an essential one, given the multiple retrieval possibilities from a complex data base), its implementation in micro-*Planner* has been sharply criticized. The problem arises from the fact that backtracking in micro-*Planner* is too automatic: the programmer cannot easily control when and where it is to occur, or in what order the search is to be carried out. See, *Sussman & McDermott,* supra note 73; *Fahlman,* A Planning System for Robot Construction Tasks, 5 Artificial Intelligence 1, 26–29 (1974). The opposing philosophy of *Conniver* is to put these decisions in the hands of the programmer directly. See, *D. McDermott & G. Sussman,* supra note 73. Emerging from this work on *Planner* and *Conniver* and the other early languages are some general ideas about how to implement these flexible control structures (backtracking, multiprocessing, etc), and future languages will undoubtedly contain more powerful and efficient mechanisms for these purposes. See, e. g. *Bobrow & Wegbreit,* A Model and Stack Implementation of Multiple Environments, 16 Comm. of the Ass'n for Computing Mach. 591 (1973); *Bobrow & Wegbreit,* A Model for Control Structures for Artificial Intelligence Programming Languages, Proceedings of the 3rd Int'l Joint Conf. on Artificial Intelligence 246 (1973); *Bobrow & Raphael,* supra note 73, at 156–58, 162-64.

84 For some examples of the range of problems that can profitably be analyzed in terms of a pattern-matching process, see *J. R. Anderson & G. Bower,* supra note 21, at 237–82, 355–78; *Moore & Newell,* How Can *Merlin* Understand? in Knowledge and Cognition 201, 219–22, 229–39 (L. Gregg, ed., 1974); *Rieger,* Conceptual Memory and Inference, in Conceptual Information Processing 157 (*R. Schank,* ed., 1975); *Wilks,* A Preferential, Pattern-Seeking Semantics for Natural Language Inference, 6 Artificial Intelligence 53 (1975).

a piece-of *S*, and Phellis owns *P*." This suggests that we mimic the structure of the English sentence by using the four *Goal* statements to locate the object *P1* and place it in the proper position inside the *Value* assertion. The following *Prog* command will have the desired effect:

```
(PROG (S P)
    (GOAL (ISSUE NEW-JERSEY ?S))
    (GOAL (STOCK ?S))
    (GOAL (PIECE-OF ?P ?S))
    (GOAL (OWN PHELLIS ?P))
    (ASSERT (VALUE ?P 5000))  )
```

By the time this program reaches the final *Assert* command, of course, the variable *P* will be assigned the value *P1* by virtue of the pattern-matching search with *Issue, Stock, Piece-of* and *Own,* and the final command will thus be interpreted as: *(Assert (Value P1 5000))*. The program thus represents quite naturally the assertion that "the value of the New Jersey stock owned by Phellis is $ 5000."

A far more significant application of the *Prog* pattern-matching capability, however, lies in the construction of abstract concepts. In the present context, this notion of an *abstraction* is quite modest – it refers only to those concepts which can be built up from a combination of elements already present in a given data structure. We have already seen one example of this: the stockholder relationship was not expressed directly in the sample dat structure of Figure 2, but it was possible to combine the concepts of *Issue, Stock, Piece-of* and *Own* to express the proposition that "Phellis is a stockholder of New Jersey." This example, however, never really developed a representation for the abstract concept of a *Stockholder;* it only developed a means of establishing one particular stockholder relationship between two particular objects, *Phellis* and *New-Jersey.* What we need now is a method for establishing the relationship *(Stockholder ?O ?C)* for any objects *O* and *C*, and this can be done by incorporating the basic mechanisms of the *Prog* command inside a structure called – for reasons that will soon become clear – a *Theorem.*

The *Theorem* is the basic device in the TAXMAN system for expressing one concept in terms of several other component concepts, and it has the following form: (*Theorem Abstract* ⟨list of variables⟩ ⟨conceptual pattern⟩ ⟨sequence of operations⟩). For example, the concept of a *Stockholder* can be represented as follows:

```
(THEOREM ABSTRACT ( O C S P)
  (STOCKHOLDER ?O ?C)
    (GOAL (ISSUE ?C ?S))
    (GOAL (STOCK ?S))
    (GOAL (PIECE-OF ?P ?S))
    (GOAL (OWN ?O ?P))  )
```

It will be noted that the main body of this theorem is identical to the main body of the *Prog* example described above, except that the constant object *Phellis* has been replaced by the variable *O*, and the constant object *New-Jersey* has been replaced by the variable *C*. Intuitively, the theorem states that the concept of

a *Stockholder* is abstractly equivalent to a particular pattern of the concepts *Issue, Stock, Piece-of* and *Own.* But the theorem also describes how the *Stockholder* concept can be *deduced* from these component parts. Since the theorem incorporates the mechanisms of the *Prog* command, it shows how the relationship *(Stockholder ?O ?C)* can be established in any given network, for any objects *O* and *C*, by the familiar *Prog* pattern-matching process – assigning variables temporarily, backtracking, etc. The only difference between the use of this theorem and the use of the *Prog* command in our original example is the fact that the stockholder variable, *O*, and the corporation variable, *C*, can now be assigned initially to any objects we choose.

To see more clearly how this conceptual definition works, let us look at the way it interacts with the various *Goal* commands examined earlier. The purpose of the definition, of course, is to permit a search for the *Stockholder* relationship as a high-level *Goal* without having to specify all the lower-level searches through the data structure. Suppose we wish to establish, as before, that "Phellis is a stockholder of New Jersey," but we know that the stockholder relationship has already been abstractly defined and stored in memory along with the usual factual network. We could then execute the command (*Goal (Stockholder Phellis New-Jersey) Abstract),* which would proceed as follows. First, the program would search the data base for a direct match with *(Stockholder Phellis New-Jersey),* but this search would obviously fail. Second, since the word *Abstract* was present in the command, the program would search for an abstraction theorem with a final deduction which matched the list *(Stockholder Phellis New-Jersey),* in this case retrieving here the definition of *Stockholder* with its matching pattern *(Stockholder ?O ?C).* Third, having successfully matched the patterns, the program would assign to the variables *O* and *C* the values *Phellis* and *New-Jersey,* respectively, then execute the four *Goal* commands of the theorem as an ordinary *Prog.* The net result, of course, would be a successful determination that "Phellis is a stockholder of New Jersey," as before. The other variations in the *Goal* commands would operate analogously. For example, if *O* were an unassigned variable, the command *(Goal (Stockholder ?O New-Jersey) Abstract)* would apply the *Stockholder* definition with one variable assigned and one unassigned, and thus would eventually succeed by matching the unassigned variable O to one of New Jersey's stockholders, maybe Phellis, maybe someone else. Similarly, the command *(Find All (Stockholder ? New-Jersey) Abstract)* would locate all of New Jersey's stockholders and return a list of all of these stockholder relationships[85].

85 The abstraction theorem just described is referred to in micro-*Planner* as a *Consequent* theorem, *G. Sussman, T. Winograd & E. Charniak,* supra note 72, and in *Conniver* as an *If-Needed* method, *D. McDermott & G. Sussman,* supra note 73, and it can be used for all types of deductive inferences. I have relabeled it here with the term *Abstract* to emphasize its principal function within the TAXMAN system. The use of the *Goal* command together with the *Abstract* theorem, as described in the text, is often referred to as a process of "pattern-directed function invocation," and it is generally considered to be one of the more important innovations of the *Planner* family of languages. See, *Bobrow & Raphael,* supra note 73 at 159, 166–69, 170. The example in the text also illustrates what is referred to as the "procedural representation of knowledge," since

Finally, let us consider a very different application of a *Prog*-like mechanism inside a *Theorem* structure, one that is essentially the reverse of the deductive process just described. Notice that the *Stockholder* definition was used in the previous example as a means of proceeding from the detailed level of the original network up to a less detailed, more abstract level expressing only a single general relationship between a corporation and its stockholders. Shouldn't there also be available a mechanism to reverse this procedure? In many situations, we may want to assert as a general proposition that "Phellis is a stockholder of New Jersey" without worrying about the detailed representation in terms of *Stock*s, *Piece*s, and *Own*ership relations, and yet we may want to have these concrete details available later for other purposes. Couldn't a program be developed to generate the low-level structures automatically?

What we want is something like:

```
(THEOREM EXPAND (O C . . .)
  (STOCKHOLDER ?O ?C)
  ⟨body of ASSERTions⟩)
```

which would be called into effect by a pattern-matching search from the command *(Assert (Stockholder Phellis New-Jersey) Expand).* I have been deliberately vague about the main body of this theorem, however, since the necessary details are more complex here than they were in the previous deductive use of the *Stockholder* definition. Basically, the program should store the four assertions – *(Issue New-Jersey ?S), (Stock ?S), (Piece-of ?P ?S)* and *(Own Phellis ?P)* – which constitute the definitional *expansion* of *(Stockholder Phellis New-Jersey),* but since the abstract *Stockholder* concept obscures all information about the values of the variables *S* and *P*, there may be more than one concrete expansion possible. The program must therefore examine the context of whatever network already exists, and make the best strategic choices possible. If there have been no assertions stored yet at all about New Jersey stock, it would be reasonable to create a class of stock for Phellis to own shares of. If the existence of one class of stock has already been asserted, it may be reasonable to list Phellis as one of the owners. If two or more classes of stock are outstanding, however, the assertion is clearly ambiguous, and it may be best simply to note the ambiguity explicitly in the network. In any event, it is clear that any decision made here could be contradicted by later assertions, and the program should be capable of backing up later on to correct such mistakes. Of course, these difficulties are not peculiar to computer data structures: they are inherent in the ambiguities of abstract concepts[86].

---

the *Stockholder* theorem there not only expresses an abstract equivalence between two conceptual structures, but also provides a concrete computer program for establishing that equivalence in a particular network. See, e. g., *T. Winograd,* supra note 20, at 108–10; *C. Hewitt,* supra note 72; *Hewitt,* Procedural Embedding of Knowledge in *Planner,* Proceedings of the 2d Int'l Joint Conf. on Artificial Intelligence 167 (1971). For a discussion of the arguments for and against these procedural representations, see *Winograd,* Frame Representations and the Declarative/Procedural Controversy, in Representation and Understanding 185, 186–93 *(D. Bobrow & A. Collins,* eds., 1975).

86 The expansion theorem just described is referred to in micro-*Planner* as an *Antecedent* theorem, *G. Sussman, T. Winograd & E. Charniak,* supra note 72, and in *Conniver* as an

The important point to emphasize here, though, is this basic process of abstracting and expanding concepts. There is not much power in a system that stores and retrieves single lists and single propositions, one at a time. But as soon as the system can represent the relationship between two levels of conceptual abstraction, its power increases substantially. It is then possible to query the system at the higher level with a (*Goal* ⟨conceptual pattern⟩ *Abstract*) command and have the lower level searches carried out automatically; or to state a fact at a higher level with an (*Assert* ⟨conceptual pattern⟩ *Expand*) command, and have the lower-level representations – subject to the caveat about ambiguity – generated automatically, too. It is also possible, of course, to extend this process of conceptual abstraction to more than one level: the *(Goal (Stockholder ?O ?C) Abstract)* statement might thus appear as a component part of some other abstract conceptual definition, and so at least two levels of data base searches could then be carried out automatically. Continuing in this way, a full abstraction hierarchy could be developed, of any complexity desired. Or multiple abstraction hierarchies could be developed, extending upwards in different directions and for different purposes from a common data base.

Perhaps even more interesting in the present context is the possibility of expanding the conceptual hierarchy downwards below the level of the concepts that have so far been treated as primitive. For example, The *Stock* predicate has been used all along in this discussion as a basic, unanalyzed term; but obviously the concept of a *Stock* is a complex one, requiring for many purposes a much more thorough elaboration. This is quite feasible in the TAXMAN system. The concept of a *stock* could itself be represented as an abstraction built up from a description of the rules governing the relationship between a *Stockholder* and its *Corporation* – a description of the stockholder's voting rights, for example; or the rights to receive dividends; or the rights to a distribution of assets upon liquidation. If this were done, a potential circularity could develop – note that the *Stockholder* concept was defined above in terms of a pattern involving the concept of *Stock;* while the proposed expansion of *Stock* might include a pattern involving the concept of *Stockholder* – but it would be a harmless circularity. In fact, the full use of such a "recursive definition" could even prove fruitful, for it could serve to point out rather clearly the fact that "primitive concepts" are to be considered "primitive" only in a relative sense. A basic philosophy of the TAXMAN systems is thus to treat concepts as primitives only provisionally, and only for certain purposes, and to retain the option of expanding them further in other contexts, as in the suggested expansion of *Stock.* It should be emphasized at this point, however, that the TAXMAN system in its current version does not expand *Stock* in this way

---

If-Added method, *D. McDermott & G. Sussman,* supra note 73. Again, I have relabeled it here with the term *Expand* to emphasize its principal function within the TAXMAN system. For a discussion of the problems of expanding a relatively abstract concept to a more concrete level in a particular context, and numerous examples of this process, see *E. Charniak,* Towards a Model of Children's Story Comprehension, AI Memo No. TR–266 (PhD. Dissertation, Mass. Inst. of Technology, 1972); *Rieger,* supra note 84; Wilks, supra note 84; *J. R. Anderson & G. Bower,* supra note 21, at 406–11.

– *Stock* is treated throughout the current version, in fact, as primitive. But the potential for expansion is there.

One final observation about these techniques for semantic information processing seems necessary here. The reader familiar with modern symbolic logic will have noted at several points in the preceding discussion that there are strong similarities between these formalisms and those of the classical predicate calculus. Indeed, the conventions for the *representation* of a concept in TAXMAN are precisely those of a higher-order predicate logic, and the use of the term *Theorem* to characterize one of the fundamental operations in the system certainly suggests that there is lurking here something like a formal method of proof. But, in fact, the *procedures* for the manipulation and utilization of these representations are quite different from those of symbolic logic, since they have the full power and flexibility of a general programming language. One difference that is often pointed out in the literature is the fact that a programming language, unlike a method of proof in symbolic logic, can express heuristic rules to guide the search for relevant theorems and propositions[88]. Another difference is the fact that the programming language can be used to modify conceptual structures in various ways, thus simulating a dynamic change in the data base, or creating a model of a hypothetical situation[89]. By contrast, the conceptual structures in the predicate calculus are fixed and timeless entities, appearing only as the passive constituents of a proof. A more subtle difference arises from the emphasis in a system like TAXMAN on the abstraction and expansion of concepts. Although the abstraction hierarchy can certainly be represented in the predicate calculus, it is not of much importance in the actual process of proving theorems; most proofs in the predicate calculus, as in mathematics, tend to be intricate and elegant deductions carried out at the upper levels of a system of very compact and parsimonious concepts. In a system like TAXMAN, though, the deductions of a *Goal* command tend to be quite different: because of the basic convention of pragmatic nominalization, discussed earlier, there tends to be a proliferation of objects in the data base; conceptual definitions tend, therefore, to be tied closely to these objects, instead of being characterized more abstractly and axiomatically; and the most important operations tend to be searches up and down the abstraction hierarchy for objects and concepts matching specified sets of definitional patterns, with the search process tightly controlled by the abstraction hierarchy itself. In this respect, the *Stockholder* concept, though overly simple, provides a typical example of the process, and shows how natural it can be. In general, the programming language used in TAXMAN seems much better adapted to the practical problems of representing and manipulating conceptual structures than the formalisms of symbolic logic[90].

88 See, e.g., *T. Winograd,* supra note 20, at 108–10; *Winograd,* supra note 85, at 190–91.

89 See, e.g., *T. Winograd,* supra note 20, at 116–17; *Winograd,* supra note 85, at 189–90.

90 There have been a number of attempts to apply formal symbolic logic to the analysis of legal concepts, all of which are subject to the comments made in the preceding paragraph. Much of this work has focused on the most abstract concepts possible – "right," duty, "privilege," etc. – drawing on the important early analysis of *Hohfeld,* Some Fundamental Legal Conceptions as Applied in Judicial Reasoning, 23 *Yale L. J.* 16 (1913); *Hohfeld,* Fundamental Legal Conceptions as Applied in Judicial Reasoning, 26

There remains the fact, however, that the TAXMAN system is abstractly – or statically – equivalent to the classical higher-order predicate calculus, and to the extent that this representation is itself a source of limitations, the TAXMAN system will certainly be subject to these limitations, too. The extent of these limitations, if any, and how they might be overcome, will be one of the issues taken up in Part Two of this article. Until then, we will push forward with an investigation of how far the general programming language by itself can take us in describing and analyzing the taxation of corporate reorganizations.

IV. Implementations

The reader who has worked diligently through the previous two sections of this article should now be able to understand easily the current implementation of TAXMAN. The TAXMAN system is a working computer program, of the sort described in Section III, which can analyze an actual case involving the taxation of

---

Yale L. J. 710 (1917). See, e.g., *Anderson,* The Logic of Norms, 1 Logique et Analyse 84 (1958), reprinted in Communication Sciences and Law: Reflections from the Jurimetrics Conference 69 (*L. Allen & M. Caldwell,* eds., 1965); *Anderson,* Logic, Norms and Roles, 4 Ratio 36 (1962); *Cullison,* Logical Analysis of Legal Doctrine: The Normative Structure of Positive Law, 53 Iowa L. Rev. 1209 (1968); Anderson, The Logic of Hohfeldian Propositions, 33 U. Pitt. L. Rev. 29 (1971); *Mullock,* Holmes on Contractual Duty, 33 U. Pitt. L. Rev. 471 (1972); *Allen,* Formalizing Hohfeldian Analysis to Clarify the Multiple Senses of 'Legal Right': A Powerful Lens for the Electronic Age, 48 S. Cal. L. Rev. 428 (1974). None of this work, however, has paid much attention to the problems of connecting up these higher-level abstractions to the lower levels of the conceptual hierarchy, preferring instead to derive conclusions by the formal manipulation of the abstractions alone. The TAXMAN system, by contrast, avoids the dizzying heights of "right," "duty," and "privilege," and focuses instead on the middle ranges of the conceptual hierarchy of corporate reorganization law; but within this middle range, the TAXMAN system is primarily concerned with representing the connections between the various levels of conceptual abstraction. There is another line of work in the literature which might appear, at first glance, to do the same thing, especially since many of the examples in this work are also drawn from the Internal Revenue Code. See, e.g., *Allen,* Symbolic Logic: A Razor-Edged Tool for Drafting and Interpreting Legal Documents, 66 Yale L. J. 833 (1957) (applications to §§ 74 and 117); *Allen & Orechkoff,* Toward a More Systematic Drafting and Interpreting of the Internal Revenue Code: Expenses, Losses and Bad Debts, 25 U. Chi. L. Rev. 1 (1957) (applications to §§ 162, 165 and 166); *Allen,* Beyond Document Retrieval Toward Information Retrieval, 47 Minn. L. Rev. 713 (1963) (applications to §§ 2031 and 2041); *Allen,* Usefulness of Modern Logic to the Readers and Writers of Legal Documents, in Communication Sciences and Law: Reflections from the Jurimetrics Conference 83 (*L. Allen & M. Caldwell,* eds., 1965) (application to § 48(a)). However, the formalism used in this work is solely that of the propositional calculus, which fails to have the expressive power of the full predicate calculus, let alone the additional advantages of a programming language. In this formalism, for example, the simple *Stockholder* concept discussed in the text could not even be represented. Two of the recent attempts to develop systems for computer-assisted legal analysis, *Popp & Schlink,* JUDITH: A Computer Program to Advise Lawyers in Reasoning a Case, 15 Jurimetrics J. 303 (1975), and Note, A Computer Method for Legal Drafting Using Propositional Logic, 53 Texas L. Rev. 965 (1975), are also based on the propositional calculus, and thus subject to the same comments.

corporate reorganizations, of the sort described in Section II. It is thus a straightforward application of the basic principles of semantic information processing to a specific area of the law. With this background firmly established, then, I will present in this section a general overview of the current TAXMAN system, followed by a detailed discussion of some of its more important features.

I have already provided several hints about how a program like TAXMAN might be written. Recall the emphasis in the preceding section on the abstraction and expansion of conceptual structures. The simple *Stockholder* program analyzed there exhibits several features that any program for semantic information processing ought to possess: most significantly, it illustrates how to represent the relationship between two or more levels of conceptual abstraction, and how to search through this abstraction hierarchy to locate the objects and concepts that match a particular definitional pattern. But, as a first approximation, at least, the analysis in Section II of the taxation of corporate reorganizations could be viewed in very much the same way. There, the various levels of conceptual abstraction consist of the lower-level "factual" descriptions of a case and the higher-level "legal" conclusions that are drawn from it. In our example, the New Jersey-Delaware hypothetical and its variations are described in terms of simple transfers of stocks and assets, simple stock distributions and stock redemptions, and so on, all at a relatively low level of abstraction; the legal conclusions are then described in terms of the concept of "control"[91], the various statutory concepts of a "reorganization"[92], and the judicial concepts of "continuity of interest"[93], "business purpose"[94] and "step transaction"[95], all at a higher level of abstraction. In these terms, the most important operation in this example consists of a search through the abstraction hierarchy to match a conceptual structure at one level to a conceptual pattern at another level, as when the New Jersey-Delaware hypothetical is said to "fit the general pattern" of both a Type C and a Type D reorganization[96]. Given this particular view of the process of legal analysis, then, the computer science models of conceptual hierarchies should be applicable rather directly to the conceptual hierarchies of corporate reorganization law.

Put in the most general way, this is precisely what the current version of TAXMAN looks like. The TAXMAN system is capable of accepting a *description* of a corporate reorganization case at roughly the level of the statement of facts in an appellate opinion; it is then capable of producing an *analysis* of the case at a somewhat higher level of abstraction, or roughly the level of abstraction of the desired legal conclusion. In doing this, the program makes use of both the

91 § 368(c). See p. 316 supra.
92 § 368(a). See pp. 316–318 supra.
93 Cortland Specialty Co. v. Commissioner, 60 F. 2d 937 (2d Cir. 1932), cert. den., 288 U.S. 599 (1933); Pinellas Ice & Cold Storage Co. v. Commissioner, 287 U.S. 462 (1933). See pp. 318–320 supra.
94 Gregory v. Helvering, 293 U.S. 465 (1935), aff'g, 69 F. 2d 809 (2d Cir. 1934). See pp. 316/317, 320/321 supra.
95 Helvering v. Elkhorn Coal Co., 95 F. 2d 732 (4th Cir. 1938), cert. den., 305 U.S. 605 (1938). See pp. 321 supra.
96 See p. 316/317 supra.

expansion and the *abstraction* mechanisms examined in the previous section. In general, the descriptions are received at a slightly higher level of abstraction than the program requires, so the expansion mechanisms are used to rewrite these descriptions at a lower level, in more concrete detail. Then, once the descriptions are fully expanded, the abstraction mechanisms are used to apply several higher-level legal concepts to them, if this is at all possible.

One should be cautious in relying on this very general characterization of the TAXMAN system, however. For the actual performance of the system obviously depends on our ability to give a precise and programmable content to these mechanisms for the "description" and "analysis" of a corporate reorganization case. If the TAXMAN system is described very loosely, and at a very high level of generality, it is possible to imagine it engaging in a rather sophisticated form of "legal reasoning". Thus, as suggested earlier[97], we might imagine the construction of multiple abstraction hierarchies, corresponding to the multiple legal characterizations of a given situation; or we might imagine the construction of multiple expansions within the concrete network, corresponding to the multiple ambiguities of any legal or factual proposition. Legal argument might then be seen as a process of manipulating these hierarchies – setting forth an abstract characterization of the law that fits a given set of facts, or a concrete expansion of the facts that fits a given abstract characterization of the law, or a complex combination of these two processes taken together. At this level of generality, too, it is possible to imagine the program handling even the more unruly concepts of corporate reorganization law. Thus even the concepts of "continuity of interest", "business purpose" and "step transaction" might conceivably be located high in the abstraction hierarchy above a densely expanded network of facts. The limitation on this kind of speculation, though, lies in the necessity of actually constructing such a conceptual hierarchy, and actually representing it within the formalisms of the TAXMAN system. In fact, the current version of TAXMAN makes no attempt to achieve such a level of complexity. It is restricted both in the range of facts that it can handle, and the range of legal concepts that it can apply. Its performance is accordingly quite limited, at least by the standards of human tax lawyers: the current version of TAXMAN manages to classify a slightly simplified description of a corporate reorganization case as either a Type B, a Type C or a Type D reorganization[98], but makes no attempt to analyze the case in terms of the more complex concepts just mentioned.

This rather modest achievement should be placed in perspective, however, and evaluated in terms of the purposes of the work as a whole. I consider it most important at this point to demonstrate that certain legal concepts *can* be modeled within the current paradigm of semantic information processing; to show in some detail *how* they can be modeled within this paradigm; and thus to show by inference how the existing program can be readily *extended* to handle problems of substantially greater complexity. The remainder of this section is devoted primarily

97 See pp. 335/336 supra.
98 See pp. 353–358 infra.

to these purposes: I will explain here how the more important description and analysis mechanisms in TAXMAN are actually constructed, and how they might possibly be improved; and I will attempt to do this in a way that invites easy generalization.

I have suggested briefly in the previous section some of the reasons for the power and flexibility of the current paradigm in semantic information processing, but it will be helpful to see illustrations of this power and flexibility in the program itself. For example, one of the claims made for the power of a programming language is that it can be used to modify a conceptual structure in various ways, thus avoiding the static qualities of the predicate calculus[99]. In Subsection IV(A), then, I will show how the TAXMAN system makes use of this capability to represent an "event" as the modification of the description of a "state," and how these event descriptions can themselves be assembled into conceptual hierarchies to represent more complex events. In Subsection IV(B), focusing on the concept of a "stock distribution," I will continue this analysis of complex event descriptions, and I will illustrate also how the TAXMAN system can take advantage of repetition and redundancy in building an abstraction hierarchy. In Subsection IV(C), finally, I will consider the implementation of the "reorganization" definition itself, showing how the pattern-matching process can be constructed and constrained, and indicating what effects these constraints might have. Some of the details presented in these subsections are rather complex, but they are necessary, I think, to develop a better sense of the capabilities and the limitations of the current program.

A.) States and Events.

One clear defect of the simple example discussed in Section III was the absence of any mechanism for describing temporal relationships. It was possible in that example to represent a simple corporate structure at any given point in time, and it was possible to *Assert* and *Erase* the propositions in the structure to indicate certain modifications over time, but there was no way to preserve the entire history of a corporate transaction in such a way that the program could analyze, temporally, what had happened to it. And yet it is obvious that the full history of a corporate transaction must somehow be made available in the TAXMAN system, if any of the basic concepts of corporate reorganization law are to be applied.

The solution to this problem is, first, to provide a mechanism in the system to indicate that a proposition is true only at a certain time, or only in a certain state. The current TAXMAN program provides the desired mechanism in the form of an optional extra variable that can be inserted at the end of each proposition that is referred to by an *Assert,* an *Erase* or a *Goal* command. The proposition *(Own Phellis P1 T4),* for example, means that "Phellis owns *P1* at time *T4*"; the same proposition without the final variable, *(Own Phellis P1),* means that "Phellis owns *P1*" at all times during the historical period being described. Conceptually, we can imagine that the data base contains a separate network, or a separate *state*

99 See pp. 336 supra.

description, for each time period under consideration. Actually, for reasons of computational efficiency, the data base contains only one network, but each proposition in the network has attached to it a separate list describing the time periods during which it is "true," and the *Assert, Erase* and *Goal* commands have been modified to check these listings of time-dependent truth values whenever necessary[100].

The second part of the solution can be found in the mechanisms in TAXMAN for representing *event descriptions.* Intuitively, an event is a transformation from one state description to another. For example, the event described by the proposition "New Jersey issues a certain class of stock" can be viewed as a transformation from a state description in which New Jersey has no such class of stock outstanding to a state description in which it does – that is, a state description in which *(Issue New-Jersey ?S)* is true for the given class of stock *S.* Actually, the event descriptions in the TAXMAN system play a dual role. First, the function simply as *representations* of the actions that have occurred. They thus have a hierarchical structure analogous to the conceptual hierarchies that have been seen already in the descriptions of static corporate structures: certain simple events are taken as primitive, and more complex events are built up systematically from complex patterns of these primitives. But the event descriptions also function as *programs* or *procedures* that actually carry out the transformation from one state description to another, physically altering the computer data structures to correspond symbolically to the alteration of states. For example, if the New Jersey corporation has no stock outstanding in a particular state description at a particular time, and if the subsequent event description represents the issuance by New Jersey of a certain class of stock, the TAXMAN system would interpret this event description as a program and write out a modified network for the subsequent time period in which, at a minimum, a proposition of the form *(Issue New-Jersey ?S)* would be asserted as true. In terms of the discussion in Section III, the event description would be *expanded* by a command of the form *(Assert* ⟨event⟩ *Expand),* with the expansion theorem now operating across an interval of time, and producing as its expansion the description of a modified state[101].

---

100 In order to implement these state description mechanisms in TAXMAN, I have actually rewritten the standard micro-*Planner* interpreter for the *Assert, Erase* and *Goal* commands. Other languages, such as *Conniver,* provide a general *Context* mechanism which allows the user to maintain multiple state descriptions and to represent easily the relationships between them. See, e.g., *D. McDermott & G. Sussman,* supra note 73; *J. Rulifson, J. Derkson & R. Waldinger,* supra note 73. For a discussion of the advantages of a general *Context* mechanism of this sort, see *Fahlman,* supra note 83, at 27. For some further examples of the possible conventions for representing time-dependent descriptions, see *T. Winograd,* supra note 20, at 116–17; *D. A. Norman, D. Rumelhart & the LNR Research Group,* supra note 21; *Rieger,* supra note 84, at 178–84.

101 For some further examples of the representation of an event as the modification of a state description, see *T. Winograd,* supra note 20 at 116–17; *Hendrix, Thompson & Slocum,* Language Processing Via Canonical Verbs and Semantic Models, Proceedings of the 3rd Int'l Joint Conf. on Artificial Intelligence 262 (1973). For a general discussion of event descriptions, see *Bruce,* Case Systems for Natural Language, 6 Artificial Intelligence 327 (1975).

Putting these state descriptions and event descriptions together, it is possible to write out in a very simple way the full history of a corporate transaction. First, we write out an initial state description; second, we write out a list of event descriptions, arranged in chronological order; third, we expand the event descriptions, in order, thus producing a full sequence of modified state descriptions. In this way, given certain assumptions about the possibility of providing a list of discrete and sequential events[102], the TAXMAN system can have available in its data base information not only about current states and current events, but about all prior states and events as well; and if the event descriptions can be written with enough complexity and diversity built into them, the histories that are thereby produced should be quite adequate for the application of the concepts of corporate reorganization law.

To see in greater detail how the event descriptions operate, let us examine a particular class of events: those involving the transfer of ownership from one individual to another. At the more complex end of this conceptual hierarchy there will be sales, dividends, stock distributions, stock redemptions – events of obvious importance for the description of corporate reorganizations. But we will consider the lower levels of the hierarchy first, beginning with the primitive concept *Trans.* For present purposes, the simplest event in the system will take the form (*Trans ?S ?X ?O ?R ?T),* which is interpreted to mean "*S* transfers *X* from *O* to *R* at time *T.*" The variable *S* represents the subject of the verb, or the actor of the event; X represents the object of the verb, or the property being transferred; *O* is the former owner; *R* is the new recipient; and *T* is the time at which the action is completed. Or, more mnemonically, the proposition can be written as: (*Trans* ⟨subject⟩ ⟨object⟩ ⟨owner⟩ ⟨recipient⟩ ⟨time⟩). Notice that *trans* refers here only to a transfer of "ownership," not to a transfer of "possession," or a transfer of "location," or any other sense of the word "transfer"[103].

The problem now is to define an expansion theorem for the *Trans* proposition. By definition, this expansion requires the modification of the ownership relationships from a prior to a subsequent state, and the essential aspects of the *Trans* theorem can thus be written as follows:

```
(THEOREM EXPAND (S X O R T TØ=(LAST STATE))
  (TRANS ?S ?X ?O ?R ?T)
```

102 This is obviously a serious restriction, and I will suggest some feasible modifications of this part of the system at pp. 371 infra. For a general approach to the problem, see *Bruce,* A Model for Temporal References and its Application in a Question Answering Program, 3 Artificial Intelligence 1 (1972).

103 It is possible to develop a variety of *Trans* propositions, as in the system of Schank and his collaborators. *Schank,* Conceptual Dependency Theory, in Conceptual Information Processing 22, 40–67 (*R. Schank*, ed., 1975). The *Trans* proposition used in the TAXMAN system is closesf to Schank's *Atrans,* or "abstract transfer," which usually refers to a transfer of possession. Schank also uses *Ptrans*, which is a basic physical act referring to the transfer of location of any physical object; and *Mtrans,* which is a basic mental act referring to the transfer of information. With these three *Trans* propositions, plus several more "primitive actions," Schank is able to represent a surprising number of English verbs.

```
(IF (GOAL (OWN ?O ?X ?TØ))
  THEN (ERASE (OWN ?O ?X ?T))
       (ASSERT (OWN ?R ?X ?T))) )
```

To see how this program works, suppose we are given the initial network of Figure 2, *supra* and suppose we wish to assert and expand the proposition "Phellis transfers the property *P1* to Delaware." The desired event can be represented by the command: *(Assert (Trans Phellis P1 Phellis Delaware* ⟨time⟩*) Expand)*. Recall from the discussion in Section III how to trace through the effects of such a command: the basic *Trans* proposition would be added to the factual data base first; then the *Trans theorem above would be retrieved and the variables matched; then the main body of the theorem would be executed with O* as *Phellis, X* as *P1, R* as *Delaware,* and with *TØ* and *T* representing, respectively, the prior and subsequent states. In the execution of the theorem, the *If . . . Then* statement would carry out the main substantive operation: the *If* clause would check to see if *Phellis* owned *P1* in the prior state; if so, as in the present case, the *Then* clause would proceed to erase *Phellis'* ownership and assert in the subsequent state description that *Delaware* now owned *P1* instead. In short, stripping away the technical details, the *Trans* assertion does exactly what we would expect – it changes the proposition *(Own Phellis P1)* into the proposition *(Own Delaware P1),* and writes out the modification in the subsequent state description network.

Once the basic *Trans* operation is defined, of course, it becomes possible to assemble some of the more complicated structures involving the transfer of property, the kinds of structures what will obviously play an important role in the description of a corporate reorganization case. One pattern of *Trans* propositions that is particularly useful involves the transfer of only a portion of an individual's interest in a piece of property. Suppose, for example, that we wish to assert the proposition: "Phellis transfers 20 shares of the property *P1* to Delaware." The formalisms for representing the ownership of stock in TAXMAN have been designed to permit a partial transfer of this sort[104], but the description of the full event here requires an additional mechanism to carry out the desired operation in the data base. Intuitively, we can imagine that Phellis' interest in the property *P1* is "split" into two "pieces," with only one of these pieces transferred to Delaware. With this image in mind, the proposition can be represented as follows:

```
(PROG  (P = (GEN))
  (ASSERT (NSHARES ?P 20))
  (ASSERT (SPLITPIECE ?P P1 ⟨time⟩) EXPAND)
  (ASSERT (TRANS PHELLIS ?P PHELLIS DELAWARE ⟨time⟩) EXPAND)  )
```

In this example, *Splitpiece* is a special event description which creates a new piece of property, *P*, of size *Nshares* = 20, and adjusts the ownership relationships in the network so that Phellis owns both the original piece of stock, *P1*, reduced in size now by 20 shares, and the new piece, which can then be fully transferred. The final command: *(Assert (Trans Phellis ?P Phellis Delaware* ⟨time⟩*) Expand)* thus carries out the transfer of the 20-share piece of stock, as desired.

104 See pp. 325 supra.

B.) Description Mechanisms.

In order for the state description and event description mechanisms to provide the flexibility that is needed in the TAXMAN system, it is necessary to define event descriptions of greater complexity than the simple *Trans* assertion analyzed above. How can this be done? Consider the concept of a stock distribution. A stock distribution is an event, like *Trans,* and so should have the effect of writing out a modified state description. But it is clearly not a simple event. In fact, it could be viewed as an entire sequence of events, a series of transfers of stock from one party to another. A stock distribution thus functions as a relatively abstract concept, and the TAXMAN system should have the capability of expanding it downwards to a more concrete level. I will discuss it here as a further example of this process of conceptual expansion.

Suppose we represent the event of a stock distribution by the proposition: (*Distribute* ⟨subject⟩ ⟨object⟩ ⟨owner⟩ ⟨recipients⟩ ⟨time⟩). It will be seen that the structure of this proposition closely resembles the structure of the simple *Trans* proposition just analyzed. Intuitively, the *Distribute* proposition describes the splitting-up and the piece-by-piece transfer of a certain property, the "object," from its current "owner" to a certain class of "recipients." For example: "The New Jersey corporation distributes its Delaware common stock to its own common stockholders." Notice in this example that the object transferred is described as "the Delaware common stock owned by New Jersey," and so could be represented in the TAXMAN system as a definite description, in the manner discussed in Section III. Notice also that the class of recipients in the *Distribute* proposition is normally represented by a definite description, tŏo: in this example, the recipients are described as "the common stockholders of New Jersey." Indeed, this description of the class of recipients plays a central role in the expansion of the *Distribute* concept. For the essence of a stock distribution usually lies in the transfer of stock according to some formula – "two-shares-for-one," "pro-rata," etc. – which takes into account the proportionate ownership of the recipients in another security, and this ownership of another security usually appears as one of the defining characteristics of the class of recipients itself. If the recipients are defined as "the common stockholders of New Jersey," for example, the distribution would typically be made in proportion to each recipient's ownership of "the common stock of New Jersey" – or, speaking more colloquially, the distribution would be made to the New Jersey common stockholders "with respect to their stock." To interpret this sort of formula, then, and thus to fully expand the *Distribute* concept, the TAXMAN system would have to analyze carefully the definition of the recipient class and apply it correctly in both of these contexts.

Let us look first at the transference of property itself, however, putting aside any questions about the proper formula for a proportionate distribution. If *P1* represents the piece of property to be distributed, then the distribution to any particular recipient, *R1*, could be described by:

(ASSERT (SPLITPIECE ?P P1 ⟨time⟩) EXPAND)
(ASSERT (TRANS ⟨subject⟩ ?P ⟨owner⟩ *R1* ⟨time⟩) EXPAND)

This expression uses the same representation of a partial transfer of property that was introduced in the discussion of the *Trans* event above: *R1* does not receive the entire property *P1*, but only a piece of it. Now, given this representation of a partial transfer to a single recipient, the full distribution to all recipients could be represented simply by repeating the partial transfer a number of times, once for each recipient. It is now relatively easy to define the expansion of *Distribute*. The expansion program for *Distribute* simply examines the description of the recipient class, and, for each *R* that satisfies this description, generates an appropriate pair of *Splitpiece-Trans* assertions, as above[105]. The final result is a list of concrete *splitpiece-trans* events which can then be executed sequentially to generate a modified state description.

With this basic framework for the *Distribute* concept in place, it is possible to return to the problems of representing a truly proportionate distribution. The *Splitpiece-Trans* assertions above carry no information about the size of the piece of property that is to be split off and transferred, but our original discussion of a partial transfer included an additional proposition precisely for this purpose. A full description of a partial transfer might thus be:

(ASSERT (NSHARES ?P ⟨number⟩))
(ASSERT (SPLITPIECE ?P P1 ⟨time⟩) EXPAND)
(ASSERT (TRANS ⟨subject⟩ ?P ⟨owner⟩ R1 ⟨time⟩) EXPAND)

But how would the program determine the proper number of shares for each recipient? Suppose we were told also a "distribution-rule" that gave each recipient *N* shares of the distributed stock for each *One* share of the security with respect to which the distribution was made. Since this is a common form for such a rule to take, it might be written out in a simple format, such as: (*Distribution-Rule* ⟨distribution⟩ *(N for one)*). How would this rule be applied? Recall that the expansion program for *distribute* examines the definition of the recipient class to obtain the name of a recipient, *R*, for each pair of *Splitpiece-Trans* assertions it generates. To apply the distribution-rule, then, the expansion program would simply examine this definition more thoroughly to extract also the ownership interest of each recipient in the defining security, assuming that the definition does, in fact, have this usual form[106]. A simple arithmetical calculation would then provide the proper value of *Nshares*.

---

105 To see this more clearly, consider how the recipient class in the example in the preceding paragraph would be represented by means of the definite descriptions discussed in Section III, at pp. 331/332 supra. In English, the class is: "the common stockholders of New Jersey." In the TAXMAN system, this description would appear as the pattern:
(GOAL (ISSUE NEW-JERSEY ?S))
(GOAL (STOCK ?S))
(GOAL (COMMON ?S))
(GOAL (PIECE-OF ?P ?S))
(GOAL (OWN ?R ?P))
The *Distribute* program would simply match this pattern to the network; it would thereby *Find All* the objects which match the variable *?R*; and it would then write out a pair of *Splitpiece-Trans* assertions for each object located.

106 This can be seen easily in the example in n. 105, supra. For when the *Distribute* program

Finally, suppose we were told that the distribution-rule is "pro-rata": (*Distribution-rule* ⟨distribution⟩ *(Prorata)).* The expansion program for *distribute* might now proceed by calculating the number *N* in the preceding *(N for One)* rule, and the applying the *(N for One)* rule as before. To do this, it would have to examine both the description of the recipient class and the description of the property to be distributed, for it would be necessary to compute the total number of shares involved in each description. However, following the general pattern of the discussion so far, this calculation would clearly be a manageable task. We are seeing here another illustration of a familiar process, the expansion of the abstract concept of a *(Prorata)* distribution in a particular context to become the more concrete concept of a *(N for One)* distribution. And we are also seeing how best to go about constructing the abstraction and expansion mechanisms themselves. As this example suggests, it is usually most profitable to start the analysis at the concrete levels and work up: when the more concrete distributional patterns are adequately understood, and adequately represented in the hierarchy, it becomes a relatively easy matter to represent the more abstract ones.

Notice now, in general, how complex this conceptual hierarchy has become, in what is still a fairly simple example. There are several levels in the hierarchy, each one capturing an important structural component of the concept of a stock distribution. There is, first of all, the structural "core" of the concept, the sequence of pairs of *Splitpiece-Trans* assertions. This structural core is then "modified," or "qualified," by the insertion of the appropriate *Nshares* propositions, all of which have been generated by the expansion of a *Distribution-Rule.* The *Distribution-Rule,* in turn, has been generated by the expansion of a still more abstract concept, such as *prorata,* describing the type of distribution in question. Finally, when the structural core and the prescribed modifications have all been assembled, they are themselves expanded to produce in the data base a modified state description network. An enormous amount of activity in the lower-level network has thus been summarized and compressed into a higher-level description, and it is this basic process of summarizing and compressing complex information which gives the conceptual hierarchy its power and flexiblity. This process is made possible, of course, by the structure of the concept of a stock distribution itself – there are recurrent patterns and recurrent features in this concept, and the TAXMAN system simply takes avantage of these recurrences to stratify and simplify the hierarchy.

C.) Analysis Mechanisms.

The two previous subsections have been concerned exclusively with the mechanisms in the TAXMAN system for the description of a corporate reorganization case: first, the mechanisms for describing simple states and events, like *Trans* and

---

matches the *Stockholder* pattern to the network to locate a recipient, *?R*, it also locates the recipient's ownership interest, *?P.* Of course, the recipient class might conceivably be defined as "the left-handed dentists in Atlantic City," in which case the search for the recipients in n. 105 supra might still succeed, given an appropriate data base, but the usual concept of a proportional stock distribution would be meaningless.

its expansions; second, the mechanisms for describing more complex events, like stock distributions. Assume, then, that the full history of a corporate reorganization case has been described in this manner, and stored in its expanded form in the data base. The problem now is to define the mechanisms that can carry out an analysis of the case in terms of the operative concepts of the Internal Revenue Code. There are some obvious contrasts between these analysis mechanisms and the description mechanisms just discussed – the description mechanisms are built up out of *Assert* commands and expansion theorems, for example, while the analysis mechanisms are built up out of *Goal* commands and abstraction theorems – but there are similarities, too, in both style and substance.

The simplest example to start with is the concept of *Control*. By statute, *Control* is defined as

> the ownership of stock possessing at least 80 percent of the total combined voting power of all classes of stock entitled to vote and at least 80 percent of the total number of shares of all other classes of stock of the corporation[107].

It should be clear why this concept is relatively easy to program into the TAXMAN system. First, the core of the concept is basically an algebraic formula, and the implementation of such a formula would appear to be a rather straightforward matter. Second, the concept is essentially static: that is, it applies only to a single state description, not to a multiplicity of states or a complex sequence of events. Finally, a pedagogical point, the concept of *Control* is composed of essentially the same elements that entered into the analysis of the concept of *Stockholder* in Section III – the idea of "stock ownership" is common to both, for instance – and this simplifies the exposition here considerably. The present analysis, therefore, need only develop a few additional variations of these earlier patterns of *Issue, Stock, Piece-of* and *Own* to fully capture the statutory definition of *Control*.

What would these variations look like? Suppose that (*control ?X ?Y* ⟨time⟩) is used to represent the proposition that "*X* controls *Y*" at a certain time. A first step in the application of this concept would be to check whether *X* owns any stock of *Y* at all, and this could be accomplished most easily with the command *(Goal (Stockholder ?X ?Y* ⟨time⟩) *Abstract)*. If the case description passes this test, the *Control* program must then determine the total stock ownership of *X* and compare this with the total amount of stock issued by *Y*. There would seem to be no way to make this comparison except by an exhaustive search. The program would therefore locate all the stock issued by *Y*, i.e.,

```
(GOAL (ISSUE ?Y ?S ⟨time⟩ ))
(GOAL (STOCK ?S ⟨time⟩ ))
```

and divide it into two classes depending on whether or not a proposition like *(Voting ?S* ⟨time⟩ *)* is true. Then, for each stock *S*, in each class, voting and non-voting, the program would locate the total ownership interest of *X*, i.e.,

```
(GOAL (PIECE-OF ?P ?S ⟨time⟩ ))
(GOAL (OWN ?X ?P ⟨time⟩ ))
```

107 § 368(c).

and calculate the total number of votes or the total number of shares, respectively, for each such *P*. Finally, sorting out the *S*'s and the *P*'s and the votes and the shares, the two separate calculations and comparisons of the "80 percent" figure would be performed in a purely mechanical fashion. Notice now how the structure of this *Control* concept resembles the structure of the *Stockholder* concept in Section III: in a sense, the *Stockholder* concept has been split in two, and several additional searches and calculations have been added to each half.

For a more complex example, consider the concept of a *B-Reorganization*. By statute, a Type B reorganization is defined as:

> The acquisition by one corporation, in exchange solely for all or a part of its voting stock (or in exchange solely for all or a part of the voting stock of a corporation which is in control of the acquiring corporation), of stock of another corporation if, immediately after the acquisition, the acquiring corporation has control of the acquired corporation (whether or not such acquiring corporation had control immediately before the acquisition)[108].

How would this definition be represented in the current formalism? Before dealing with all the details, it may be helpful first to focus on the underlying structure of the concept, taking an initial clue from the language of the statute itself. Notice that a Type B reorganization is treated in the statute as a noun, and that it is defined generically as a type of "acquisition," which is also treated as a noun. But what kind of an "object" in the case description could this noun phrase be pointing to? It could not be pointing to any of the objects that have appeared so far, certainly, all of which have been represented by specific symbols in the semantic network, e.g., *S2*, *P1*, *Phellis*, *Delaware*. Instead, the term "acquisition," viewed as a noun, must be pointing to a more complex conceptual structure in the network, a conceptual pattern which can be abstracted from its context and treated, for certain purposes, as an object itself. In this case, a likely candidate for the abstracted pattern would be a structure representing an "exchange," or a mutual "transfer" of property – that is, some particular organized structure of *Trans* propositions.

To formalize this notion, we can draw some ideas from our earlier discussion of a stock distribution. Recall the highly structured and stratified representation developed for the *Distribute* concept: there were several levels in the conceptual hierarchy in that example; there was a structural core to the concept, plus several modifiers; and there were several distinct stages of abstraction and expansion required. All of this structural complexity contributed to the economy and efficiency of the higher-level representations, of course, and the same should be true of the *Acquisition* concept, if properly designed. Assume, then, that an acquisition has the basic structure of an "exchange," or, more properly, a list of exchanges: *(Acquisition* ⟨list ⟨exchange⟩$_i$ ⟩ *)*. Assume also that an exchange has the basic structure of a pair of "transfers," or, more properly, a pair of lists of transfers: *(Exchange*⟨list ⟨trans⟩$_j$ ⟩ ⟨list ⟨trans⟩$_j$ ⟩ *)*. Putting all this together, an acquisition would be represented as follows:

108 § 368(a) (1) (B).

```
(ACQUISITION
  ⟨list  (EXCHANGE
          ⟨list (TRANS ?S1 ?X1 ?O1 ?R1 ⟨time⟩ ) . . .⟩
          ⟨list (TRANS ?S2 ?X2 ?O2 ?R2 ⟨time⟩ ) . . .⟩ )
          . . . .⟩ )
```

Although the notation in this example may be confusing, the structure itself is actually quite simple. It represents the two basic levels of the acquisition concept, as suggested in the previous paragraph, and as suggested also in the discussion of a stock distribution – i.e., it represents an acquisition as an "exchange," which is itself a mutual "transfer." The full meaning of this representation should become clearer as the discussion progresses.

With the *Acquisition* structure defined in this way, let us now examine in greater detail the application of the *B-Reorganization* concept to a typical case description. First, consider the problem of matching the *Acquisition* structure to the given network. Starting at the innermost level, the *Trans* propositions would be matched with the command: *(Goal (Trans ?S ?X ?O ?R* ⟨time⟩ *) Abstract).* But the assignment of variables in this matching process cannot be entirely unconstrained, since the *Trans* propositions here must fit within the *Exchange* structure at the next higher level, and the "recipient" of one transfer must therefore be the same as the "prior owner" of the other. To accomplish this, the *Exchange* program would require that the *O*'s and *R*'s in each list of *Trans* propositions be identical to one another, that the R_1's from the first list be equal to the O_2's from the second, and that the R_2's from the second list be equal to the O_1's from the first. Once the *exchange* structure is fully matched, then, the program obtains a single recipient, R_1, who has exchanged certain properties with another single recipient, R_2. This result will now be quite close to the final *Acquisition* structure the program is looking for, except for the possibility of an *Acquisition* consisting of multiple exchanges to a single recipient. But this last possibility can be handled easily, too. By convention, we will take R_1, the recipient in the first list of *Trans* propositions, to be the "acquirer" in the final *Acquisition:* if there is more than one *Exchange* to R_1, then, these multiple exchanges would all be located and listed in the *Acquisition* structure; if not, the *Acquisition* structure would consist of only one *Exchange.*

With the concept of an *Acquisition* defined and applied in this way, it is now a relatively simple matter to represent the remainder of the *B-Reorganization* definition: a *B-Reorganization* is just a further set of constraints on the *Acquisition* structure. For example, one constraint is the statutory requirement that the acquisition be "by one corporation," which means simply that R_1, the acquirer located by the *Acquisition* match, be a corporation, or: *(Goal (Corporation R1)).* Another simple constraint is the requirement that the acquisition be "of stock of another corporation," which means that each X_1 which is transferred to R_1 in the first list of *Trans* propositions should be a *Piece-of* a *Stock* which is *Issued* by some other *Corporation.* A more crucial constraint is the "solely for voting stock" requirement. By statute, the acquisition must be

> in exchange solely for all or a part of [the acquiring corporation's] voting stock (or in exchange solely for all or a part of the voting stock of a corporation which is in control of the acquiring corporation) . . .

To apply this test, the *B-Reorganization* program must locate all the X_2's in the second list of *Trans* propositions, i.e., all the properties transferred "in exchange for" the X_1's, and determine that these X_2's are either (a) each one a *Piece-of* a *Voting Stock* which is *Issued* by R_1, or (b) each one a *Piece-of* a *Voting Stock* which is *Issued* by another *corporation* in *control* of R_1. From the earlier discussions of these various concepts, however, it will be seen that the machinery necessary to implement this "solely for voting stock" requirement is already available: the problem now is simply one of arranging these existing conceptual structures in the appropriate patterns within the *B-Reorganization* concept. As a final example, consider the constraint that "immediately after the acquisition, the acquiring corporation has control of the acquired corporation." Again, the components of this test are all in place: the "time" of the acquisition has been recorded in the lists of *Trans* propositions; the "acquiring corporation" has been previously identified, namely, R_1; and the "acquired corporation" has been located in the course of the search for the issuer of the stock X_1. We now need only add the specification: (*Goal* (*Control* ⟨acquiring corporation⟩ ⟨acquired corporation⟩ ⟨time⟩) *Abstract*).

The preceding account has distorted the actual application of the *B-Reorganization* definition, however, in one crucial respect. I have been pretending here that the abstract structure of the concept has completely determined the order of the search procedures necessary to apply it – that is, I have been describing a particular sequence: *first* the matching of the *Acquisition* concept to the network, *then* the imposition of the *B-Reorganization* constraints. For expository purposes, this is a useful pretense: it highlights the structural core of the concept, and it provides a suggestion of the connections between the *B-Reorganization* structure and the other structures that are a part of the TAXMAN system, especially *Distribute*. But in many situations this would be a highly inefficient procedure. For example, it would be highly inefficient for the program to locate, first, a large number of *Acquisition* structures, and then, but only then, to discover that these were not acquisitions of *Stock*, or not even acquisitions by a *Corporation*. Instead, ideally, the *B-Reorganization* constraints should be checked at the same time that the *Acquisition* structure is being matched. The current version of TAXMAN takes a step in this direction, in fact, by specifying a particular order in which the *B-Reorganization* and *Acquisition* searches are to be performed: first, the program searches for a relationship of *Control* between two corporations, and locates the earliest time at which this control exists; second, it searches backwards in time to locate the transfers of stock, if any, to the controlling corporation; third, for each such transfer of stock to the controlling corporation, it locates the transfers made in exchange, and determines whether or not these are all solely for voting stock; finally, if any one of these initial decisions leads to a dead end, the program backs up to the appropriate point and starts the search over again from there. For the great majority of situations we would expect to encounter, this procedure appears to be reasonably efficient. Even so, it is not an ideal solution. A more sophisticated version of the program would be able to alter its search procedures as it goes along, depending on the specific information it is searching for and the specific

knowledge it has already obtained – but this degree of sophistication is beyond the current capacity of the TAXMAN system.

It is interesting to note at this point the various substantive legal questions that are reflected in this choice of a search procedure. There are several ambiguities in the structure of the *B-Reorganization* concept as it has been described so far: first, the *Acquisition* structure is an open-ended list of *Exchange* relationships, and it is unclear how far back into the past or forward into the future this list should be extended; second, and closely related, the *Control* relationship can obviously exist over an extended period of time in the history of a corporate transaction, and the process of obtaining control can sometimes be a lengthy one. Together, these ambiguities produce the problems of "creeping acquisitions"[109]. Is it necessary, for example, that control be obtained all at once in the tax-free acquisition itself[110]? Does a prior acquisition of stock for cash disqualify from reorganization treatment a subsequent acquisition made solely for voting stock[111]? Conversely, can a prior acquisition of stock in exchange for voting stock qualify as part of a subsequent tax-free reorganization, even if the prior acquisition occurred before the requisite control was obtained[112]? In the TAXMAN system, the answers to these

109 See, e.g., *B. Bittker & J. Eustice,* supra note 25, ch. 14, at 33–34; *Kanter,* Cash in a "B" Reorganization: Effect of Cash Purchases on "Creeping" Reorganization, 19 Tax L. Rev. 441 (1964); *MacLean,* "Creeping Acquisitions," 21 Tax L. Rev. 345 (1966).

110 Prior to 1954, the Type B reorganization was defined as: "The acquisition by one corporation, in exchange solely for all or a part of its voting stock, of at least 80 per centum of the voting stock and at least 80 per centum of the total number of shares of all other classes of stock of another corporation." Int. Rev. Code of 1939, ch. 1, § 112 (g) (1) (B), 53 Stat. 40. This language seemed to require that the entire 80 percent controlling interest in the acquired corporation be purchased in a single solely-for-voting-stock transaction, thus precluding the use of a Type B reorganization by a corporation that already owned more than 20 percent of the acquired corporation's stock. See Lutkins v. United States, 312 F. 2d 803 (Ct. Cl. 1963), cert. denied, 375 U.S. 825 (1963); Robert A. Pulfer, 43 B.T.A. 677 (1941), aff'd per curiam, 128 F. 2d 742 (6th Cir. 1942); but see Charles A. Dana, 36 B.T.A. 97 (1937), aff'd, 103 F. 2d 359 (3d Cir. 1939). The 1954 statute was intended to reverse this result, by specifying only that the acquiring corporation have control "immediately after the acquisition . . . (whether or not such acquiring corporation had control immediately before the acquisition)." Under the 1954 statute, then, a corporation which already owns 75 percent of another corporation's stock can purchase solely for voting stock an additional 10 percent, and qualify the transaction as a Type B reorganization; a corporation which already owns 85 percent can purchase solely for voting stock an additional 1 percent, and likewise qualify the transaction as a Type B reorganization. See Treas. Reg. § 1.368–2(c).

111 The Treasury Regulations permit a reorganization treatment in this situation, provided that the prior acquisition is sufficiently separate. The example given is that of a corporation making a 30 percent stock-for-cash purchase in 1939, and then a 60 percent stock-for-voting-stock purchase in 1955. Treas. Reg. § 1.368–2(c). For a discussion of some of the problems that arise here, see *MacLean,* supra note 109, at 367–72.

112 A literal reading of the 1954 statute might preclude reorganization treatment in this situation, but the Treasury Regulations allow a tax-free acquisition to occur "in a series of transactions taking place over a relatively short period of time such as 12 months." The example given is a further elaboration of the example in n. 111 supra: the acquiring corporation, on March 1, 1955, offers to exchange its own voting stock for all the stock of the acquired corporation that is tendered within 6 months from the date of the offer;

questions are determined by specifying the order and the scope of the search procedures for the *Acquisition* and *Control* concepts. If the *Acquisition* structure is located first, and then a check is made for the *Control* relationship, there is a tendency for these questions to be decided in one way; if the *Control* relationship is located first, and then the *Acquisiton* structure is assembled, there is a tendency for these questions to be decided in another way. Actually, the current TAXMAN system does a rather poor job here, since it fails to accurately embody any of the current rules on these issues[113]. The important point to note, though, is that the current formalism reveals all the possibilities quite clearly, and thus provides a means of examining in concrete detail the possible solutions. A feasibly extended version of the TAXMAN system could contain further rules – expressed in higher-level concepts – about "creeping acquisitions"[114].

and by the end of the 6 month period, the acquiring corporation thereby obtains an additional 60 percent of the stock, bringing its total to 90 percent. Treas. Reg. § 1.368–2(c). In American Potash & Chemical Corp. v. United States, 402 F. 2d 1000, modifying 399 F. 2d 1964 (Ct. Cl. 1968), the Court of Claims held that a series of stock-for-voting stock acquisitions taking place over a period of 14 months (with 48 percent of the stock acquired between September 38 and November 3, 1954, and the remaining 52 percent acquired on November 30, 1955), would not qualify as a Type B reorganization "unless the entire series is proved to have been a part of a continuing offer to purchase," id., at 1001. For a further discussion of this problem, see *Henderson*, Voting Stock in a Two-Step Asset Acquisition: the Kimbell-Diamond Reorganization, 25 Tax L. Rev. 375, 386–94 (1970).

113 The most glaring defect lies in the fact that the current TAXMAN system has no provision for decomposing a sequence of exchanges into two or more separate acquisitions; it acts as if all transfers of stock to the acquiring corporation, past and future, are part of a single transaction. This defect would be most evident if the *Acquisition* structure were located first, and the *Control* theorem applied later, as in the search procedure initially described in the text, pp. 350/351 supra. As the TAXMAN system is currently programmed, however, the control relationships are located first, and a search is then made for prior acquisitions. At first glance, this procedure might appear to implement the 1939 rather than the 1954 statute, note 110 supra, since it seems to search for an acquisition resulting in control, rather than an acquisition which is succeeded by a state description in which control merely exists. This would be the case, in fact, if the searches for prior acquisitions were only made backwards from the time of the earliest control; but the current system, after first searching backwards from the earliest control relationship, can then search forward for all the later control relationships which are themselves preceded by prior acquisitions; in this way, the system can locate an entire sequence of transactions each of which qualifies as a *B-Reorganization*. The forward search will continue until the control relationship is no longer true, or until an exchange is made which is not solely for voting stock, and this provides a simple mechanism for terminating the future extent of the *Acquisition* structure. It should be evident from this brief description, though, that the treatment of "creeping acquisitions" was not carefully thought out at the time the current TAXMAN system was designed. The effects described here are, in many instances, simply the unanticipated consequences of a number of routine programming decisions that were made in fitting the *Acquisition* and *control* structures together to make up the *B-Reorganization* concept. It is not implausible to suggest, of course, that the problems of interpretation under the 1939 and the 1954 statutes have similar origin: the unanticipated consequences of choosing a particular mode of expression for the "acquisition" of "control."

114 At a minimum, these rules could incorporate the simple Treasury Regulations, notes 111 and 112 supra, to distinguish a single acquisition from two or more separate acquisitions.

V. Results

Let us now observe the TAXMAN system in operation, as it describes and analyzes an actual case involving the taxation of corporate reorganizations[115]. The overall structure of the system should be clear from the discussion in the preceding section, but it is important to note here how this structure corresponds to the actual use of the system, at least in its current version[116]. TAXMAN is an on-line real-time computer program, and the description mechanisms and analysis mechanisms correspond quite literally to the operations that can be performed by a user of the program, sitting and typing at a terminal. The user thus types out a description of the case, in the programming language we have been working with throughout[117], starting with an initial state description and proceeding with a sequence of event descriptions[118]. The user then interrogates the system with the various *Goal* commands, bringing into play in the process the abstraction theorems[119]. In this way, the user determines whether or not – and why, or why not – the case fits the pattern of a Type B, a Type C or a Type D reorganization.

As an illustration of this, I will use the case of *United States v. Phellis*[120], one of the earliest reorganization cases, and the case upon which our original New Jersey-Delaware hypothetical was based. The basic factual pattern is thus the same as the factual pattern discussed throughout Section II – a New Jersey corporation

115 The TAXMAN system was programmed for and run on the PDP 10 computer at the Stanford Artificial Intelligence Laboratory. See notes† and 3, supra. The example of a corporate reorganization case that follows occupied a core image of approximately 40,000 words on that machine, of which approximately 20,000 words were available for the assertions and theorems necessary to describe and analyze the case. The abstraction and expansion theorems were expressed in approximately 450 lines of code in micro-*Planner* and *Lisp*, see note 72 supra; and the case description itself required slightly over 200 lines of code in addition. I did not keep careful records of the response time of the system, but the illustrations in the text generally took less than 30 seconds to process, when the time-sharing system was lightly loaded. The micro-*Planner* programming language is known to be slow and inefficient, however, see, e. g., *Sussman & McDermott,* supra note 73, and a moderate increase in the complexity of the TAXMAN system would have produced a substantial degradation of performance. Complete listings of the program and a sample terminal session are available from the author upon request.

116 There are a number of restrictions imposed by this structure, see, e. g., notes 117–19 infra, and the possibilities of escaping from some of these restrictions will be explored in Section VI, infra.

117 See notes 72–74 supra. It is very tedious to write case descriptions in the micro-*Planner* programming language, of course, and until this requirement can be relaxed, a system like TAXMAN will be accessible only to a small number of specialists. For a discussion of the possibilities of incorporating a natural language capability into a system like TAXMAN, see pp. 364–366 infra.

118 See pp. 342 and note 102 supra. For a discussion of the possibilities of relaxing this requirement, see pp. 361 and notes 135–38 infra.

119 See pp. 346–353 supra. For example, the user might set up as a *Goal* the application of the *B-Reorganization* theorem, which would in turn call into play at several points the *Control* theorem. For a discussion of alternative modes of analysis in a system like TAXMAN, see pp. 362/363 and notes 139–42 infra.

120 257 U. S. 156 (1921).

transferring its assets to a newly formed Delaware corporation in exchange for Delaware stock – but there are additional complexities in the *Phellis* case itself, since there are several classes of stocks and bonds outstanding in that case, several transfers of property, and several distributions and redemptions of securities. One of the purposes of the illustration, then, is to indicate the range of factual situations which the current description mechanisms can handle. A second purpose is to indicate the kinds of analysis the system can produce. Since the original *Phellis* case was decided under a statute which contained no provisions for a tax-free corporate reorganization, the TAXMAN system will make no attempt here to reproduce the original decision, but will instead attempt to analyze the case as it would be decided under the present Code, asking whether or not it fits into any of the present statutory categories. I have already pointed out some of the ways in which the TAXMAN analysis is imperfect and incomplete[121], and these will be confirmed by the illustration; but the illustration should also suggest the potential power of the approach. In this respect, it is particularly interesting to watch, not just the results, but the full process of applying an abstract statutory concept to a particular factual situation, to see in detail how the conceptual patterns are searched for and retrieved from the case description network.

The following is a full statement of the facts in *United States v. Phellis:*

Mr. Justice Pitney delivered the opinion of the court. The court below sustained the claim of C. W. Phellis for a refund of certain moneys paid by him under protest in discharge of an additional tax assessed against him for the year 1915, based upon alleged income equivalent to the market value of 500 shares of stock of a Delaware corporation called E. I. du Pont de Nemours & Co., received by him as a dividend upon his 250 shares of stock of the E. I. Du Pont de Nemours Powder Co., a New Jersey corporation. The United States appeals.

From the findings of the Court of Claims, read in connection with claimant's petition, the following essential facts appear. In and prior to September, 1915, the New Jersey company had been engaged for many years in the business of manufacturing and selling explosives. Its funded debt and its capital stock at par values were as follows:

5% mortage bonds	$ 1,230,000
4½% 30-year bonds	$ 14,166,000
Preferred stock ($ 100 shares)	$ 16,068,600
Common stock ($ 100 shares)	$ 29,427,100
Total	$ 60,891,700

It had an excess of assets over liabilities showing a large surplus of accumulated profits; the precise amount is not important, except that it should be stated that it was sufficient to cover the dividend distribution presently to be mentioned. In that month a reorganization and financial adjustment of the business was resolved upon and carried into effect with the assent of a sufficient proportion of the stockholders, in which a new corporation was formed under the laws of Delaware with an authorized capital stock of $ 240,000,000 to consist in part of debenture stock bearing 6 per cent cumulative dividends, in part of common stock; and to this new corporation all the assets and good will of the New Jersey company were transferred as an entirety and as a going concern, as of October 1, 1915, at a valuation of $ 120,000,000, the new company assuming all the obligations of the old except its capital stock and funded debt. In payment of the consideration, the old company retained $ 1,484,100 in cash to be used in redemption of its outstanding 5% mortgage bonds, and received $ 59,661,700 par value in debenture stock of the new company (of which $ 30,234,600 was to be used in taking up, share for share and dollar for dollar, the preferred

121 See pp. 339, 351/352 and notes 113–114 supra.

stock of the old company and redeeming its 30-year bonds), and $ 58,854,200 par value of the common stock of the new company which was to be and was immediately distributed among the common stockholders of the old company as a dividend, paying them two shares of the new stock for each share they held in the old company. This plan was carried out by appropriate corporate action; the new company took over all the assets of the old company, and that company besides paying off its 5% bonds acquired debenture stock of the new company sufficient to liquidate its 4½% 30-year bonds and retire its preferred stock, additional debenture stock equal in amount at par to its own outstanding common stock, and also two shares of common stock of the Delaware corporation for each share of the outstanding common stock of the New Jersey corporation. Each holder of the New Jersey company's common stock (including claimant), retained his old stock and besides received a dividend of two shares for one in common stock of the Delaware company, and the New Jersey corporation retained in its treasury 6 percent debenture stock of the Delaware corporation equivalent to the par value of its own outstanding common stock. The personnel of the stockholders and officers of the two corporations was on October 1, 1915 identical, the new company having elected the same officers as the old; and the holders of common stock in both corporations had the same proportionate stockholding in each. After the reorganization and the distribution of the stock of the Delaware corporation, the New Jersey corporation continued as a going concern, and still exists but, except for the redemption of its outstanding bonds, the exchange of debenture stock for its preferred stock, and the holding of debenture stock to an amount equivalent to its own outstanding common and the collection and disposition of dividends thereon, it has done no business. It is not, however, in process of liquidation. It has received as income upon the Delaware company's debenture stock held by it, dividends to the amount of 6% per annum, which it has paid out to its own stockholders including the claimant. The fair market value of the stock of the New Jersey corporation on September 30, 1915, prior to the reorganization, was $ 795 per share, and its fair market value, after the execution of the contracts between the two corporations, was on October 1, 1915, $ 100 per share. The fair market value of the stock of the Delaware corporation distributed as aforesaid was on October 1, 1915, $ 347.50 per share. The Commissioner of Internal Revenue held that the 500 shares of Delaware company stock acquired by claimant in the distribution was income of the value of $ 347.50 per share and assessed the additional tax accordingly . . .[122]

Obviously, some of these facts could not be represented at all in the current TAXMAN system. For example, it is not possible to express the fact that the New Jersey company "had been engaged for many years in the business of manufacturing and selling explosives." Nor is it possible to express the fact that the plan of reorganization was "carried into effect with the assent of a sufficient proportion of the stockholders," or that the "personnel of the . . . officers of the two corporations was on October 1, 1915 identical, the new company having elected the same officers as the old," since the current system contains no provisions for representing any of the rules and concepts related to these sorts of internal corporate decisions. But it should also be clear that the more operative facts of the case could indeed be represented in TAXMAN, given the basic description mechanisms we have examined so far.

For example, the initial state description consists of a New Jersey corporation with common and preferred stock and two classes of bonds outstanding, and one identified stockholder, Phellis, who owns 250 shares of the common stock. This initial state description is thus only slightly more complicated than the simple example discussed earlier in the text. There are several event descriptions in the

122 257 U. S. 156, 165–68 (1921).

case, too, all of them only slight variations of our earlier examples. There is a transfer of "all the assets and good will of the New Jersey company . . . at a valuation of $ 120,000,000 . . ." There is the fact that the New Jersey company receives "$ 59,661,700 par value in debenture stock of the [Delaware] company . . . and $ 58,854,200 par value of the common stock of the [Delaware] company . . ." There is a redemption of the 5 percent mortgage bonds with cash, and a redemption of the preferred stock and the 30-year bonds with debenture stock. Finally, and not coincidentally, there is the precise example used earlier as an illustration of the description mechanisms of TAXMAN: the fact that "$ 58,854,200 par value of the common stock of the [Delaware] company . . . [is] immediately distributed among the common stockholders of the [New Jersey] company as a dividend, paying them two shares of the [Delaware] stock for each share they held in the [New Jersey] company."

When the *Phellis* case is written out in full in the TAXMAN system, and expanded in full, it produces a moderately complex network. There is a list of 14 higher-level event descriptions, and there are approximately 50 internally generated symbols – all with the mnemonic prefix *Phe* – which represent the new classes of stocks and bonds, the ownership interests that are transferred, etc. Assuming, then, that the full description is expanded and stored in the data base, let us now look at the process of applying the analysis mechanisms to it. How closely does the Phellis case fit the patterns of the Type B, the Type C, and the Type D reorganizations? Suppose we query the system first with: *(Goal (B-Reorganization ?A ?C ?T) Abstract),* where *A* is taken to be the acquiring corporation, *C* is taken to be the acquired corporation, and *T* is taken to be the time at which the reorganization is complete. Since we have left all the variables in this query open, the system will search for a Type B reorganization, at any time, with either Delaware or New Jersey as the acquiring corporation. Consider first the search for: *(B-Reorganization Delaware New-Jersey ?T).* This search will fail, as it should, since Delaware never acquires any stock of New Jersey. In terms of the order of search described earlier, the failure occurs in searching for *(Control Delaware New-Jersey ?T);* and within the *Control* theorem, the failure occurs in searching for *(Stockholder Delaware New-Jersey ?T).* Consider now the search for: *(B-Reorganization New-Jersey Delaware ?T).* This search will also fail, but for a more complex reason. Here the system finds *(Control New-Jersey Delaware Phe32),* and then finds the transfers of stock that have preceded this establishment of control. The result is an acquisition structure as follows:

```
(EXCHANGE
    ((TRANS DELAWARE PHE29 DELAWARE NEW-JERSEY PHE30)
     (TRANS DELAWARE PHE31 DELAWARE NEW-JERSEY PHE32))
    ((TRANS NEW-JERSEY PHE26 NEW-JERSEY DELAWARE PHE28)) )
```

To see what the system is doing here, let us examine the identities of the objects *Phe29, Phe31* and *Phe26.* It turns out that *Phe29* and *Phe31* represent the debenture stock and the common stock, respectively, that was newly issued by Delaware and then transferred to New Jersey. Thus, curiously enough, the system

123 § 368(a) (1) (C).

has succeeded in matching a portion of the *B-Reorganization* pattern to the description of the *Phellis* case: it has found an "acquisition" of Delaware stock by New Jersey, which is followed by a situation in which New Jersey "controls" Delaware. But *Phe26* then turns out to represent the assets of Delaware, and thus in checking the "solely for voting stock" requirement of the Type B reorganization, the system discovers that the transfer is not even made in exchange for "stock", let alone "voting stock". Hence the search to match the full pattern of the *B-Reorganization* concept ultimately fails, as it should.

Suppose we now query the system with: *(Goal (C-Reorganization ?A ?C ?T) Abstract)*. By statute, the Type C reorganization is defined as:

> The acquisition by one corporation, in exchange solely for all or a part of its voting stock (or in exchange solely for all or a part of the voting stock of a corporation which is in control of the acquiring corporation), of substantially all of the properties of another corporation . . .

The implementation of this concept in TAXMAN has some similarities to the implementation of the Type B reorganization, as one might guess. When the *C-Reorganization* pattern is matched to the description of the *Phellis* case, the system finds the following acquisition structure:

```
(EXCHANGE
    ((TRANS NEW-JERSEY PHE26 NEW-JERSEY DELAWARE PHE28))
    ((TRANS DELAWARE PHE29 DELAWARE NEW-JERSEY PHE30)
     (TRANS DELAWARE PHE31 DELAWARE NEW-JERSEY PHE32)) )
```

and proceeds to apply the additional constraints of the statutory definition. Thus, the system checks first to determine that *Phe26* is "substantially all of the properties" of the New Jersey corporation, which it is. Then it examines what is exchanged for these properties: *Phe29,* which turns out to be the Delaware debenture stock, and *Phe31*, which turns out to be the Delaware common stock. If the debenture stock is non-voting, of course, the "solely for voting stock" requirement will fail. For purposes of the illustration, however, I have stipulated in the case description that both of these stocks are, in fact, voting stocks. With this stipulation, the *C-Reorganization* program succeeds, and returns: *(C-Reorganization Delaware New-Jersey Phe28).*

Finally, let us look at the concept of a Type D reorganization, defined as:

> A transfer by a corporation of all or a part of its assets to another corporation if immediately after the transfer the transferor, or one or more of its shareholders (including persons who were shareholders immediately before the transfer), or any combination thereof, is in control of the corporation to which the assets are transferred; but only if, in pursuance of the plan, stock or securities of the corporation to which the assets are transferred are distributed in a transaction which qualifies under section 354, 355 or 356[124].

The latter part of this definition cannot presently be implemented in the TAXMAN system, since there are no facilities for representing the distributions that would

124 § 368(a) (1) (D).

qualify under §§ 354, 355 or 356[125]. Nevertheless, we can implement the structural core of the remainder of the concept: the transfer of property to a corporation which the transferor then controls. When this core concept is matched against the description of the *Phellis* case, the system finds:

((TRANS NEW-JERSEY PHE26 NEW-JERSEY DELAWARE PHE28))

and ((CONTROL NEW-JERSEY DELAWARE PHE32)
(CONTROL NEW-JERSEY DELAWARE PHE33)
(CONTROL NEW-JERSEY DELAWARE PHE36))

Restricted to this partial fragment of the concept, then, the system determines that a Type D reorganization has occured, and returns: *(D-Reorganization New-Jersey Delaware Phe28).*

One interesting feature of this example, of course, is the fact that the various reorganization patterns can be matched up to the case description network here in multiple ways. Notice that the same three *Trans* propositions appear in both the *C-Reorganization* pattern and the partial *B-Reorganization* pattern, but with their order within the *Exchange* structure reversed in the two situations. Notice also in the partial *B-Reorganization* pattern that the *Control* structure appears in association with the acquisition of stock; whereas in the *D-Reorganization* pattern, the same *Control* structure appears in association with the transfer of assets. The existence of these partial matches and multiple matches suggests some of the ways in which the concepts of corporate reorganization law can be manipulated in the planning of a corporate transaction, and some of the ways in which ambiguities can arise in the analysis of a corporate reorganization case. It is potentially a very powerful feature of the TAXMAN system that it can identify and display these partial and multiple matches automatically.

VI. Feasible Extensions

As I explained in the Introduction to this article, one of the main purposes behind the work on TAXMAN was a practical one – the expectation that some of the techniques developed here could eventually prove useful in automating the more mundane aspects of legal research and analysis. It should be clear from the illustration in the preceding section, of course, that the TAXMAN system in its present form does not yet provide a very useful tool for the practicing tax lawyer. For one thing, the coverage of the system is much too limited: it is not particularly interesting to be told only the statutory classification of a slightly simplified version of a case like *United States v. Phellis.* Even with a broader coverage of the tax law, the current system would be much too cumbersome: the necessity of writing state descriptions and event descriptions in sequential order, and doing so in an obscure programming language, would render the system totally inaccessible to a potential user. Nevertheless, viewed as a simple pilot program rather than a full-scale applications program, the TAXMAN system does suggest a number of

125 See pp. 315 and notes 41 & 47 supra.

extensions which should eventually prove to be of practical significance. In this section, I will describe some of these extensions, and sketch briefly the directions in which the future work in this area might proceed. Then – assuming that the extensions I describe are, in fact, feasible – I will outline how a more sophisticated version of the TAXMAN program might function within a system for computer-aided legal research, and compare such a system with the legal research systems on the market today.

The coverage of the system, first of all, could be extended in many instances simply by a straightforward application of the basic techniques that have been analyzed throughout this article. For example, one important area for further work would be the treatment of distributions and exchanges pursuant to a statutory reorganization. Recall that the major defect of the *D-Reorganization* concept discussed in Section V resulted from the inability of the current TAXMAN system to analyze the distributions that would qualify under §§ 354, 355 or 356. An extended version of the system should be able to represent at least part of these nonrecognition rules[126], and at least part of the associated basis rules[127]. More ambitiously, an

126 See pp. 315 and notes 35–36, 41 & 47 supra. The basic structure of the nonrecognition rule could be represented, certainly, since it has some similarities to the basic conceptual structures we have already examined in Section IV, supra. For example, the general rule of § 354(a) (1) reads: "No gain or loss shall be recognized if stock or securities in a corporation a party to a reorganization are, in pursuance of the plan of reorganization, exchanged solely for stock or securities in such corporation or in another corporation a party to the reorganization." Here, the core of the concept is a simple exchange structure, which would cause very few difficulties for the TAXMAN system, and the more important problems would then arise in representing the concepts that are attached to this conceptual core. See generally, *B. Bittker & J. Eustice,* supra note 25, ¶¶ 14.30–.32, 14.34. The definition of a "party to the reorganization" in § 368(b) would not be too difficult to implement, since it is built up out of a number of definite descriptions extracted from the components of the reorganization concept itself. See, *B. Bittker & J. Eustice,* supra note 25, ch. 14 at 79–80; Treas. Reg. § 1.368–2(f); but see Groman v. Commissioner, 302 U. S. 82 (1937); Helvering v. Bashford, 302 U. S. 454 (1938). The concepts of "stock" and "securities" are not defined in the statute, but it is possible to articulate some of the rules governing the application of these concepts, as these have emerged from the cases. See, *B. Bittker & J. Eustice,* supra note 25, ch. 14 at 73–79; Treas. Reg. § 1.354–1(e); cases cited note 50 supra; Neville Coke & Chemical Co. v. Commissioner, 148 F. 2d 599 (3d Cir. 1945), cert. denied, 326 U. S. 726 (1945); Carlberg v. United States, 281 F. 2d 507 (8th Cir. 1960). Not all the components of the general nonrecognition rule could be handled in a straightforward fashion, however. The concept of an exchange being made "in pursuance of the plan of reorganization" is not well defined at all, and it is further complicated by its involvement in the problems of "creeping acquisitions", pp. 315 supra, "business purpose", pp. 320/321 supra, and "step transactions", pp. 351/352 supra. See, *B. Bittker & J. Eustice*, supra note 25, ch. 14 at 80, 98–103; *Manning,* "In Pursuance of the Plan of Reorganization": The Scope of the Reorganization Provisions of the Internal Revenue Code, 72 Harv. L. Rev. 881 (1959); Treas. Reg. §§ 1.368–1(c), –2(g), –3(a). The difficulties in representing these ill-defined concepts will be discussed further in Part Two of this article, but it may be useful to provide a brief preview of that discussion at this point. In Part Two, I will characterize the conceptual structures that appear in the current version of TAXMAN as logical templates, a term intended to suggest the form of the upper-level concepts occurring in all the conceptual hierarchies we have seen so far. As an alternative, I will attempt to

extended version of the system should be able to represent some of the rules for distributions generally, even outside the reorganization context[128]. It seems clear that an extension of the analysis mechanisms in these directions would require a comparable extension of the description mechanisms: the system would have to expand the concepts of stocks and bonds, as suggested earlier[129], and provide a formalism for describing various hybrid securities[130]; the system would also have

describe how an upper-level concept can take the form of a similarity cluster, which consists of a prototype conceptual structure plus a number of deformations of the prototype in the lower-level network. The more ill-defined concepts will then be analyzed as similarity clusters, rather than logical templates, and certain dynamic properties of these similarity clusters will be suggested. A full exposition of these ideas, with references to the relevant literature, will have to be postponed until Part Two of this article. For the remainder of the present article, then, the discussion will be confined to the more straightforward generalizations of the TAXMAN paradigm.

127 See p. 315/316 supra. The general rule of § 358(a) is quite mechanical: "the basis of the [nonrecognition] property . . . received [is] the same as that of the property exchanged, decreased by the fair market value of any other property [and] any money received by the taxpayer, . . . and increased by . . . the amount of gain to the taxpayer which was recognized on such exchange . . ." Thus, assuming that the nonrecognition rules could be represented well enough to determine "the amount of gain . . . recognized", the representation of the basis rules would be a simple matter. If a stockholder receives several different stocks and securities, and exchanges several different stocks and securities, there are further rules prescribing the allocation of basis, but these, too, could be represented quite easily. See, Treas. Reg. § 1.358–2. See generally, *B. Bittker & J. Eustice*, supra note 25, ch. 14 at 93–95.

128 See pp. 313/314 supra. The rules for corporate distributions have some of the same characteristics that were observed in note 126 supra: some of these rules could be handled in the TAXMAN system in a very straightforward fashion, but others might require a fundamental modification of the TAXMAN paradigm. For example, the rules for identifying "stock redemptions" include the simple mechanical tests of a "substantially disproportionate" distribution, § 302(b) (2), as well as the vague standard of a redemption "not essentially equivalent to a dividend", § 302(b) (1). The new stock dividend rules of § 305 specify certain distributions which are taxable, §§ 305(b) (3)–(5), but also require an inquiry into whether or not a distribution "has the result of the receipt of property by some shareholders and an increase in the proportionate interests of other shareholders", § 305(b) (2). The general dividend rule, § 316(a), requires a determination of the corporation's "earnings and profits", which can be computed in most instances from the complex rules of § 312 and Treas. Reg. § 1.312, but which is nowhere given a comprehensive statutory definition. See generally, *B. Bittker & J. Eustice*, supra note 25, ¶¶ 7.01–04, 7.60–.63, 9.20–.24.

129 See pp. 335/336 supra.

130 A more expansive description of the various types of securities would be necessary, for example, to implement the rules governing the exchange of "stock or securities" under § 354 (a), see note 126, supra, and to implement the rules governing "stock dividends" in § 305, see note 128 supra. The general approach here would be to develop descriptions of the possible components of a security interest: the voting rights, the priorities upon liquidation, the dividend provisions, etc. These components could then be combined in various combinations to produce any number of exotic securitites: "convertible subordinated debentures," see, e.g., § 279 (b); "hermaphrodite convertible preferred stock," see, e.g., Rev. Rul. 70–108, 1970–1 C.B. 78; or whatever. For an indication of the complexity of this area, see *B. Bittker & J. Eustice*, supra note 25, ¶¶ 4.02–.09; *Plumb*, The Federal Income Tax Significance of Corporate Debt: A Critical Analysis and a Proposal, 26 Tax. L. Rev. 369 (1971).

to provide a better formalism for corporate accounting[131], and for some of the economic consequences of the transactions that occur[132]. But once this expanded factual network is developed, a good many of the rules and concepts governing corporate distributions generally, and reorganization exchanges specifically, could be represented in abstraction theorems not too dissimilar to those of the current version of TAXMAN[133]. These extensions would not be trivial, of course; they would require many hours of careful work; but they would not seem to require any major conceptual breakthroughs[134].

It is not quite so straightforward a problem to overcome the rigidities of the current system, and produce a program that is less cumbersome to operate, but there are a number of techniques available to handle these difficulties, too. I have already mentioned some of the difficulties that arise from the requirement that event descriptions be written out in a strict sequential order[135]: an extended version of the TAXMAN system should have built into it a more sophisticated model of temporal relationships[136], and a method of manipulating these relationships by

131 The formalism for corporate accounting would be necessary to implement any of the rules concerning "earnings and profits" of §§ 312 and 316 (a), see note 128 supra, and in a number of other contexts.

132 For example, a very simple economic model would be necessary to represent the concept of "gain" or "loss", and to implement the basis rules of § 358, see note 127 supra. More complex models would be required to represent the economic differences between the ownership of various classes of securities, as in note 130 supra. Finally, for a fuller understanding of the "purposes" and "motivations" behind the various reorganization patterns, and an understanding of the divergences between "form" and "substance", it would be necessary for the TAXMAN system to have available a means of describing both the tax and the nontax benefits to the participants in any corporate transaction. The development of these economic models would not be a simple task, but once developed they would play an important role in an extended version of TAXMAN.

133 It is interesting to observe that the proposals in notes 126–28 supra would, in effect, treat the *Distribute* concept as an analysis mechanism, and implement it as an abstraction theorem, whereas the previous discussion of corporate distributions, pp. 344–346 supra, treated the *Distribute* concept as a description mechanism, and implemented it as an expansion theorem. It should be possible to exploit this duality of representation to achieve greater efficiency and economy in the conceptual hierarchy, as suggested at pp. 348/349 supra.

134 There are two caveats to this statement. The first has already been indicated in notes 126–28 supra: although many of the concepts and rules of corporate reorganization law could be implemented in an extended version of TAXMAN, as suggested, there are others that would surely elude the current methods of semantic information processing. The second caveat has to do with the sheer size of the extended TAXMAN system: it may be possible in principle to describe the factual situations and conceptual structures that occur in this area of the law, but the programming effort could conceivably become bogged down in the complexity of the details. A similar reservation is expressed about the prospects for natural language communication with TAXMAN, at pp. 364–366 and note 148 infra. Both of these caveats will be discussed further in Part Two of this article. My own judgment, however, is that a large portion of corporate reorganization law could be incorporated into an extended version of TAXMAN before either of these limits is reached.

135 See pp. 342 and note 102 supra.

136 The current version of TAXMAN represents "time" as a simple linear list. The natural

means of higher-level descriptions[137]. More generally, an extended version of TAXMAN should permit a more flexible use of description and analysis mechanisms of all kinds. One rigidity of the current system is the requirement that a case be described in full, and then expanded in full, before any analysis mechanisms can be applied. A more flexible system would accept a partial description, produce only a partial expansion, and attempt with this to do a partial analysis; the analysis mechanisms would then generate, if necessary, an additional expansion, or request from the user an additional piece of the description[138]. A still more radical variation would shift the operation of the entire system from an "analysis" mode to what might be called a "planning" mode. Notice that the current system is designed to produce a legal analysis of a given sequence of events, as if these

---

generalization would use an unordered set of "time points", which could then be given a partial ordering by relationships such as "before" and "after"; and a corresponding set of "time intervals", which could be connected to each other by expressions such as "A occurs during B", and "B overlaps with C". It should be possible in this system to express incomplete knowledge, or even to express inconsistent knowledge, about these temporal relationships. For an example of a model with these properties, see *Bruce*, supra note 102; see also *Rieger*, supra note 84. For an extensive philosophical analysis, see *A. N. Prior,* Time and Modality (1957); *A. N. Prior*, Past, Present and Future (1967); *A. N. Prior*, Papers on Time and Tense, (1968).

137 As an ultimate goal, an extension of the TAXMAN system in this direction would permit the description of a case in whatever order seemed most natural, using either explicit or implicit temporal references to translate this natural description into the underlying temporal model. For an example, consider the factual description of United States v. Phellis, at pp. 354/355 supra. The narrative in that case follows a rough chronological order, but there are many points at which it backs up in time to add further details, or to add a new perspective, and then jumps ahead again to carry the description up to the present. The temporal relationships are kept straight by the use of tenses (e.g., "which was to be used in taking up", "which was to be and was immediately distributed"), by the use of explicit temporal references (e.g., "after the reorganization and the distribution"), and by the use of other more subtle contextual clues. The design of a computer system that can handle this much complexity, with this much flexibility, is an enormously difficult problem, however. The most that can be hoped for in the near future is a rather modest relaxation of the very rigid requirements of the current TAXMAN system.

138 The problems that arise in designing a system with these characteristics are similar to the problems that arise in designing a system for more flexible temporal references, note 137 supra, only more severe. One characterization of the issue here is in terms of a store vs. recompute decision: is it preferable to expand all factual descriptions at once, and thus "store" in computer memory everything that can be said about a given situation? or is it preferable to leave the descriptions in an unexpanded form, as they are received, and then "recompute" the lower-level information every time an analysis mechanism actually needs it? Any response to this issue, however, requires a response to a large number of other issues about the eficient representation of conceptual information in computer data structures, and this leads quickly to most of the currently active areas of research in semantic information processing. For a systematic discussion of these problems in terms of a list of "design issues" that must be faced in constructing any representation of conceptual knowledge, see *Moore & Newell,* supra note 84; *Bobrow,* Dimensions of Representation, in Representation and Understanding 1 (*D. Bobrow & A. Collins,* eds., 1975). Some of these issues have been briefly mentioned earlier in this article: see, e. g., the discussion of control structures, note 83 supra; pattern-matching, note 84 supra; abstraction theorems and procedural representations, note 85 supra; expansion theorems, note 86 supra; and references cited therein.

events have already occurred; in a planning mode, by contrast, the system would construct a sequence of events, with a given end result, so as to satisfy a given legal analysis[139]. Although this is not a trivial problem, it is one that has been solved in other contexts[140]. In general, if the conceptual structures in a problem domain are reasonably well understood, and reasonably well defined, it is a manageable task to devise a computer program to search heuristically for an appropriate "plan". In TAXMAN, for example, it should be possible to reproduce some of the formal manipulations that were seen in *Helvering v. Eklhorn Coal Co.*:[141] in other words, given the initial state description in that case, and a description of the desired end result, an extended version of the TAXMAN system should be able to notice that the Type C reorganiziation almost works, except for the problem of "substantially all properties", and then with this information generate the sequence of Type D, Type C, and Type B reorganiziatons that would be, formally, tax-free[142].

139 This distinction between an analysis mode and a planning mode plays an important role in the article by *Buchanan & Headrick,* supra note 4, where the authors distinguish between two models of creative legal research. "In the first model, the factual situation has been set, certain events have happened, and a client wants to know what his rights, remedies and risks are. In the second, a client is looking for guidance for a future action, and the client can control the facts or events somewhat to suit his legal needs as assessed by the lawyer." Id., at 47. It should be noted, of course, that these two models are not entirely distinct, and that each one can appear as a component of the other. It is obvious that the planning mode requires the use of the analysis mode to determine the legal consequences of various alternative plans; it is less obvious that the analysis mode may often require the use of the planning mode as well. In the corporate tax area, for example, the arguments concerning the proper legal characterization of a given transaction may be strongly influenced by a consideration of the alternative ways in which the transaction could have been carried out: this can be seen in many of the cases involving the "business purpose" doctrine, pp. 320/321 supra, and the "step transaction" doctrine, pp. 320/321 supra. The representation of this kind of an argument is beyond the current capability of the TAXMAN system, but it will be discussed in Part Two of this article.

140 A close analogy can be seen in the problem of chemical synthesis, where computer programs have been written to construct a sequence of chemical reactions from commercially available compounds that will produce a desired organic molecule, such as Vitamin A. See, *Sridharan,* Search Strategies for the Task of Organic Chemical Synthesis, Proceedings of the 3rd Int'l Joint Conf. on Artificial Intelligence 95 (1973); Corey, Computer-Assisted Design of a Complex Organic Synthesis, Science (Oct. 1:, 1969). Other examples include: constructing proofs of simple mathematical theorems, see, e.g., *Bledsoe, Boyer & Henneman,* supra note 23; *Gelernter,* Realization of a Geometry-Theorem Proving Machine, in Computers and Thought (*E. Feigenbaum & J. Feldman,* eds., 1963); building towers and other structures in a world of children's blocks, see, e.g., *Winograd,* supra note 20; *Fahlman,* supra note 83; *Fikes, Hart & Nilsson,* Learning and Executing Generalized Robot Plans, 3 Artificial Intelligence 251 (1972). For some of the earlier theoretical work on the problems of generating "plans" of this sort, see *McCarthy,* Situations, Actions and Causal Laws, in Semantic Information Processing 410 (*M. Minsky,* ed., 1968); *Amarel,* On Representations of Problems of Reasoning About Actions, 3 Machine Intelligence 131 (*D. Michie,* ed., 1968); *Simon,* On Reasoning About Actions, in Representation and Meaning: Experiments with Information Processing Systems 414 (*H. Simon & L. Siklossy,* eds., 1972).

141 95 F. 2d 732 (4th Cir. 1938), cert. denied, 305 U.S. 605 (1938); see pp. 321 supra.

142 It is interesting to observe that this proposal for generating a "plan" would, in effect,

Finally, let us look briefly at the problem of natural language understanding, since most readers will perceive correctly that this is one of the major obstacles to a practical application of the TAXMAN system. The important point to note here is that the type of programming language in which TAXMAN is written is intended to function as part of a larger system for natural language processing, and thus it would be possible – though not at all easy – to write programs that could translate automatically from the description and analysis mechanisms of TAXMAN into a simplified form of English, and vice versa. The best example of a program of this sort is found in the work of Winograd, cited earlier[143]. Winograd's system conducts a dialogue in English about a simple world of blocks – small yellow cubes, large red boxes, tall green pyramids, etc. – which can be moved around and stacked up on a table top. The system has three main components which interact with each other: a component for *inference,* which maintains the basic model of the block world, and is written in the same language as TAXMAN[144]; a component for *syntax,* which contains a basic grammar of English[145]; and a component for *semantics,* which mediates between the other two components[146]. Put very simply, Winograd's system operates by translating an English sentence, usually a question or a command, into the internal representation of the block world model, generating an appropriate response within this model, and then translating this response back into English. The dialogues that are thereby produced can be remarkably complex, so long at least as the topic of conversation is confined strictly to the

---

treat the REORGANIZATION concept as a description mechanism, and implement it as an expansion theorem; whereas the previous discussion of corporate reorganizations, pp. 348–352 *supra,* treated the REORGANIZATION concept as an analysis mechanism, and implemented it as an abstraction theorem. The planning mode can thus be viewed as a process of positing, hypothetically, a higher-level description that we would like to have applied to our case, and then generating the lower-level expansions which would make that hypothetical description come true, consistent with the other assertions already present in the data base. On the dual representation of such concepts, compare note 133 *supra.*

143 See notes 20, 74, 85, 88–89, 100–01, 140 supra.

144 *Winograd,* supra note 20, at 23–27, 108–26.

145 Id., at 16–23, 46–108. The syntactic component is based on *Halliday,* Notes on Transitivity and Theme in English, 3 J. of Linguistics 37 (1967) and 4 J. of Linguistics 179 (1968); *Halliday,* Functional Diversity in Language as Seen from a Consideration of Modality and Mood in English, 6 Foundations of Language 322 (1970).

146 *Winograd,* supra note 20 at 28–34, 126–69. *Winograd* provides the following introductory account of the role of the semantic component in his system: "There has never been a clear definition of what the field of 'semantics' should cover, but attempts to program computers to understand natural language have clarified what a semantic theory has to do, and how it must connect with the syntactic and logical aspects of language. In practical terms, we need a transducer that can work with a syntactic analyzer, and produce data which is acceptable to a logical deductive system. Given a syntactic parser with a grammar of English, and a deductive system with a base of knowledge about particular subjects, the role of semantics is to fill the gap between them." Id., at 28. In many ways, the semantic component is the key to the success of *Winograd's* system, and the reader is advised to consult the complete work for an understanding of how it operates.

world of blocks[147]. The obvious suggestion, of course, is that a program of this sort be used in the TAXMAN system, with the topic of conversation confined strictly to the world of corporate reorganizations. In the long run, this will certainly be possible; but in the short run, the attempt would probably be stymied by the problem of complexity. Even in its very restricted current version, the TAXMAN system is somewhat more complex than the block world model, mainly because of the complexity of the higher-level concepts of "distributions" and " reorganizations". A future version of TAXMAN, extended in coverage and flexibility as suggested above, would greatly exceed the capacity of the current programs for the syntactic and semantic components of natural language[148]. I consider it more important, then, to develop the other feasible extensions first, and to restrict communications with the computer for the time being to a high-level programming language, as in the current system. At some point in the future, though, when the extended version of TAXMAN has been fully developed, it should be possible to

---

147 The following dialogue is typical. The system has been requested earlier to "stack up both of the red blocks and either a green cube or a pyramid," and it has done so, in the process removing a small pyramid from the top of a red cube so that it could use that cube to complete the stack as specified. The following exchange then occurs (with queries to the system given in lower case, and responses in upper case):
Had you touched any pyramid before you put the green one on the little cube?
YES, THE GREEN ONE.
When did you pick it up?
WHILE I WAS STACKING UP THE RED CUBE, A LARGE RED BLOCK AND A LARGE GREEN CUBE.
Why?
TO GET RID OF IT.
Why did you do that?
TO CLEAN OFF THE RED CUBE.
Why did you clear off that cube?
TO PUT IT ON A LARGE GREEN CUBE.
Why did you do that?
TO STACK UP THE RED CUBE, A LARGE RED BLOCK AND A LARGE GREEN CUBE.
Why did you do that?
BECAUSE YOU ASKED ME TO.
How did you do it?
BY PUTTING A LARGE RED BLOCK ON THE TABLE; THEN LETTING GO OF IT; THEN PUTTING A LARGE GREEN CUBE ON IT; THEN LETTING GO OF THAT CUBE; THEN PUTTING THE RED CUBE ON THAT CUBE; THEN LETTING GO OF THAT CUBE.
How many objects did you touch while you were doing it?
FOUR OF THEM.
What did the red cube support before you started to clean it off?
THE GREEN PYRAMID.
There were five blocks to the left of the box then.
NO, ONLY FOUR OF THEM: THE RED CUBE, TWO LARGE GREEN CUBES AND A LARGE RED BLOCK.
. . .
Id., at 13–14.

148 For a discussion of the problem of complexity in building a natural language system, see *Simmons & Bennet-Novak,* Semantically Analyzing an English Subset for the Clowns Nicroworld, Am. J. Computational Linguistics, mic. 18 (1975).

return successfully to the design of a natural language processor for corporate reorganization law.

Let us now take a somewhat speculative view of the proposals here, and imagine that the extensions of TAXMAN described in the preceding praragraphs have, in fact, been achieved. How might this program function within a system for computer-aided legal research? The basic idea would be to develop and maintain a data base consisting of: (1) a large number of factual descriptions, taken from reported cases, Revenue Rulings, Treasury Regulation hypotheticals, etc.[149]; (2) a wide variety of legal concepts and legal rules, taken from the Code, the Regulations, and other sources[150]; and (3) for each factual description in the data base, the set of all possible legal characterizations of these facts in terms of the stored concepts and rules[151]. If the system is sophisticated enough, it should be possible to develop this data base in a natural way: each factual description could be stored and analyzed in the same way that the case of *United States v. Phellis* was stored and analyzed, but with greater descriptive flexibility[152], and with some form of natural language input[153]. The system could then be used in several ways. First, it could be used simply as a retrieval device: it would be relatively easy to retrieve the factual situations which match certain patterns of interest, or which satisfy certain fragments of a legal concept or a legal rule. Second, it could be used to develop a suggested analysis of a new case: as in the *Phellis* example, again, the user could describe the new case to the system and then generate a range of applicable higher-level concepts. Finally, as suggested earlier, the system could be used in a planning mode: the user could describe an initial situation, and a desired end result, and then generate a number of possible transaction patterns with the desired tax consequences[154]. If all of these various facilities could be successfully integrated into a single computer program, of course, it would be a powerful program indeed[155].

It may be instructive to compare this imagined system with the systems for computer-aided legal research and analysis which are currently available. There are, first of all, a number of systems in operation today which maintain a data base of statutes and decided cases, and which retrieve this information in response to an appropriate query[156]. But the form in which these materials are stored, and the

149 See, e.g., notes 129–32 supra.
150 See, e.g., notes 126–28 supra.
151 See e.g., the discussion of partial and multiple matches, at pp. 358, 364–366 supra.
152 See pp. 361/362 and notes 135-38 supra.
153 See pp. 364–366 and notes 143-48 supra.
154 See pp. 368/369 and notes 139–42 supra.
155 But see the caueats in note 134 supra.
156 The two most prominent systems in this country are LEXIS, developed by Mead Data Central, Inc., and WESTLAW, a service recently offered by the West Publishing Co. Other important legal retrieval systems include: LITE, an early system developed by the Air Force; JURIS, developed by the Justice Department; QUIC/LAW, developed at Queen's University in Canada; DATUM, developed at the University of Montreal; UNIDATA in Switzerland; CREDOC in Belgium; and CRIDON in France. The ancestor of most of these retrieval systems was the early work of Horty at the University of Pittsburgh Health Law Center, which subsequently became the Aspen Systems Corporation. For a collec-

techniques for retrieval, are quite different from those of the proposed extension of TAXMAN. The existing retrieval systems store the full text of a legal document and retrieve it by key-words, or combinations of key-words, which in its most primitive form amounts simply to requesting, e.g., "all cases in which the word *jurisdiction* appears"[157]. There is no denying that this type of retrieval can be very useful in many circumstances, but even at its best it is confined to looking only at what might be called the "surface statistics" of legal language. By contrast, the extended version of TAXMAN would store a representation of the underlying conceptual structure of a statute or a case, and retrieve this structure by a sophisticated pattern-matching operation[158]. Assuming, then, that the TAXMAN system

---

tion of presentations by the developers of many of these systems, see American Bar Ass'n, Automated Law Research (1973). For a more thorough survey and analysis, see *C. Tapper*, Computers and the Law (1973); *P. Slayton*, Electronic Legal Retrieval (Canadian Dep't of Communications, 1974); *Sprowl*, Computer-Assisted Legal Research, 1 ABF Research J. 175 (1976); *Bigelow*, The Use of Computers in the Law, 24 Hast L. J. 707 (1973); *Comment*, Computer-Assisted Legal Research, 51 Ore. L. Rev. 665 (1972); *Kayton*, Retrieving Case Law by Computer: Fact, Fiction and Future, 35 Geo. Wash. L. Rev. 1 (1966).

157 A typical example of the retrieval process in the LEXIS system is given in Harrington, What's Happening in Computer-Assisted Legal Research? 60 A.B.A. J. 924, 930–31 (1974). Harrington poses the problem of finding a case in the United States Supreme Court which addresses the following question: "If the United States appears in a probate court to assert a claim against a decedent's estate, does it thereby waive its immunity against being sued in a state court so that the fiduciary can assert a counterclaim greater than the amount of the original claim asserted by the United States?" The first request to the LEXIS system is to search for all cases in which the phrase "United States" occurs within ten words of either the word "counterclaim" or the word "crossclaim", or in the search language of the system: *United States W/10 counterclaim or crossclaim.* The computer locates 25 cases which satisfy this request, and the researcher decides to modify the search by specifying that the case should also contain the phrase "state court". This time the computer locates 19 cases, and the researcher decides to specify further that the word "jurisdiction" be found within seven words of the phrase "state court". In the search language of the system, the query is now: *United States W/10 counterclaim or crossclaim and state court W/7 jurisdiction.* The computer this time locates only one case which satisfies this last request, and the researcher decides to display it on the screen. It is United States v. Shaw, 309 U.S. 495 (1940), which happens to be precisely on point. The researcher can then "shepardize" the Shaw case automatically, by searching for all subsequent cases in which the Shaw citation appears. The use of the West Publishing Co.'s WESTLAW system is similar, except that the data base consists of West headnotes rather than the full text of the opinions. See, *Sprowl*, The WESTLAW system – a different approach to computer-assisted legal research, 16 Jurimetrics J. 142 (1976).

158 As a simple illustration of the difference between these two approaches, consider the pattern-matching search that was described in Section V, supra. Suppose we wish to locate a case in which there occurs a single transaction fitting the general pattern of a Type C and a Type D reorganization, and almost fitting the pattern of a Type B reorganization, except for the "solely for voting stock" requirement. In an extended version of TAXMAN, this specification could easily be expressed, and the system would retrieve the case of United States v. Phellis, among others. But this search request could not even be formulated in the existing legal retrieval systems, nor is it likely that any other request could be formulated which could retrieve with precision the factual pattern of the Phellis case. The difficulty in locating the Phellis case itself is obvious: the

can be extended as suggested to cover a larger range of factual situations and a larger portion of the concepts of corporate reorganization law – and this is obviously a crucial assumption – it should prove to be substantially more powerful and flexible than the systems for legal information retrieval on the market today[159].

The second approach to computer-aided legal research today is based upon the methods of computer-assisted instruction (CAI), and it has some functional similarities to the "analysis" and "planning" modes of the suggested TAXMAN system. The approach can be seen in its clearest form when it is used as an educational tool: here the computer functions as a tutor, posing questions and hypothetical

---

statutory reorganization provisions were inapplicable to that case, and the opinion contains no mention of them. Even confining the search to cases that post-date the reorganization statutes, the existing retrieval systems would still encounter difficulties. The most efficient way to search for a Type C reorganization, for example, is to search for the statutory section number "368(a) (1) (C)," and its predecessor in the 1939 Code, "112(g) (1) (C)." Thus a researcher might request a search for 368(a) (1) (B) AND 368(a) (1) (C) AND 368(a) (1) (D), and then a search for 112(g) (1) (B) AND 112(g) (1) (C) AND 112(g) (1) (D). But the system would then retrieve any case in which these three statutory sections were mentioned, whether applicable or not, and no case which failed to discuss all three; there would be no way to specify to the system that the three reorganization patterns should fit the same basic transaction, or that the Type B reorganization pattern should only be partially applicable. For example, had it been decided under the 1939 or the 1954 statute, one case that would be retrieved with this request is Helvering v. Elkhorn Coal Co., 95 F. 2d 732 (4th Cir. 1938), see pp. 321 supra, where the three reorganization patterns apply to sequential transactions, rather than a single transaction. Although this illustration is slightly artificial, the point it makes is generally valid, and similar illustrations could be constructed for most other kinds of retrieval problems in the corporate tax area.

159 A similar conclusion is advanced in *P. Slayton,* Electronic Legal Retrieval (Canadian Dep't of Communications, 1974). Slayton investigated several of the current legal retrieval systems (especially DATUM and QUIC/LAW, the Canadian systems) and analyzed their performance within a framework of tentative assumptions about how lawyers actually think, and how they actually conduct research. He noted especially the disparity between the search strategies employed by these computerized retrieval systems, and the kinds of research strategies customarily employed by practicing attorneys. His conclusions were sharp: "electronic legal retrieval, if widely embraced, may distort legal thinking, may affect unfavorably important features of the legal system such as the doctrine of precedent and the law-making ability of judges, and may enhance existing social inequalities. One point is clear. Large resources, whether human, financial, or whatever should not be devoted to the maintenance or development of electronic legal retrieval systems of the existing type without further extensive study." Id., at 25. One area of further study that *Slayton* recommended was the investigation of artificial intelligence techniques ". . . to determine whether and in what respects they constitute a line of development worthier of pursuit than development of established retrieval systems." Id. For a response to Slayton's study by the Director of the DATUM project at the University of Montreal, see *MacKaay,* Book Review, 15 Jurimetrics J. 108 (1974); for a response by the President of Mead Data Central, Inc., the developer of the LEXIS system, see *Rubin,* Fear and Trembling in the Groves of Academe, 15 Jurimetrics J. 112 (1974); see also, *Slayton,* A Short Reply to MacKaay and Rubin, 15 Jurimetrics J. 115 (1974). In a subsequent study for the Canadian Department of Communications, *Slayton* analyzed in greater detail the prospects for the application of artificial intelligence techniques in the law, *P. Slayton,* Radical Computer Use in Law (Draft Report prepared for the Canadian Dep't of Communications, June, 1974), and concluded, not surprising-

cases to a law student, and evaluating the responses[160]. Used as an aid to the practicing attorney, the computer would pose questions designed to elicit the essential facts of a client's case, and then, depending on the information received, it would either suggest a tentative legal analysis or pose an additional factual question[161]. There are parallels here, of course, to the way an extended version of TAXMAN might develop the description and analysis of a case like *United States v. Phellis.* But note the rigid structure of these CAI-based systems: the user of the CAI system faces a series of pre-programmed questions, and usually responds in a strict multiple-choice format; the author of the program faces the tedious task of anticipating all possible responses, and programming explicitly every logical branch[162]. In the extended version of TAXMAN, by contrast, the most important

---

ly, that the existing computer science methods were incapable of capturing the full complexity of legal reasoning, thus raising many of the issues that will be dealt with in Part Two of this article. Nevertheless, Slayton recommended that a limited research effort be undertaken, id. at 82–86, applying some of the more straightforward artificial intelligence techniques to the problem of defining the concept of a "principal residence" in the capital gains provisions of the Canadian Income Tax Act, § 40 (2) (b), Can. Stat. c. 63 (1970–71–72).

Slayton's ultimate conclusions and recommendations are thus very much in accord with those of the present article.

160 Most of the programs for computer-assisted legal instruction in this country have been written by Professor *Park* of the University of Minnesota Law School, and Professor *Keeton* of the Harvard Law School. There is not an extensive published account of this work, but one recent survey article is Landis, Teaching Law with Computers: Workshop Report. As of September, 1976, there were thirteen exercises available, most of them in Civil Procedure and Evidence, and these are now being offered to interested law schools through the *Educom* computer network. For a through survey of computer-assisted instruction generally, see *R. Levien,* The Emerging Technology: Instructional Uses of the Computer in Higher Education (1972); for a sharp critique, see *A. Oettinger & S. Marks,* Run, Computer, Run (1969).

161 Although this approach has not yet been extensively developed, there are reports in the literature of several pilot projects along these lines. One such project, initially funded by OEO Legal Services, and then spun off as a private corporation, has produced a client interviewing system for use by the Cook County Legal Assistance Foundation in Chicago. See, *Chatterton & McCoy,* Computer-Assisted Legal Services, 1 L. & Computer Tech., Nov., 1968, at 2; R. *McCoy, W. Chatterton & R. Knitter,* Computer-Assisted Legal Services Project: Final Report (Int'l Computer Associations, Inc., Madison, Wisc., 1971); N. Y. Times, Feb. 17, 1974, at 83. Other participants in the Cook County project have reportedly produced a similar system for estate planning and administration, which they plan to market commercially. See, Directory of Jurimetrics Projects, 15 Jurimetrics J. 56 (1974). The same approach would appear to be applicable to the business of computerized tax return preparation, but most of the firms in this business still require their clients to fill out printed questionnaires by hand and send them in by mail. See generally, *Hutner,* Taxes and Time-Sharing, 49 Taxes 140 (1971); Computer-Prepared Returns: Companies Again on the Increase, but Watch Out for "Weak Sisters", 35 J. Taxation 358 (1971).

162 A typical example of one of the CAI legal exercises is given in Landis, supra note 160, at 10. The computer begins by printing out the following fragment of a trial transcript, and noting the point at which an objection might be raised: "Q. Did you exchange papers with defendant? A. Yes. Q. What did he put on his paper, if anything? Objection 4 (If permitted, the witness would answer): 'He put his name and address, his insurance company's name, and the statement "Jones Car Repair is at fault for not fixing the

and difficult work is done at the start, when the description and analysis mechanisms are designed to capture the basic conceptual structure of the problem domain; once these basic conceptual structures are written, the "analysis" and "planning" modes of the system can then be programmed more easily, in a uniform and systematic way. For the user of the system, this permits a much greater flexibility in the interactive process; and for the author of the program, this requires much less concern about the specific details of the interaction. In general, then, the TAXMAN approach would appear to have much greater long-range potential than the CAI approach as a technique for computer-aided legal research, either for the law student, or for the practicing attorney[163].

---

brakes.'''' The computer then prints out the following instructions: "Push 'S' (objection sustained; evidence excluded) or 'O' (objection overruled; evidence admitted); then push the return button." Assuming that the student pushes the "S" button, the computer responds as follows:

You have ruled that evidence is inadmissible. What are your grounds for excluding this evidence?

1. Irrelevant/prejudicial
2. Best evidence rule
3. Leading question
4. Ask for opinion
5. Hearsay
6. Privilege
7. I want to change my answer and overrule the objection.

Push a number key to give your answer; then push the return button.

163 The position taken here is similar to that of *Carbonell* and his associates, who have done the pioneering work on the application of artificial intelligence techniques to computer-assisted instruction. See, e. g., *Carbonell,* AI in CAI: An Artificial-Intelligence Approach to Computer-Assisted Instruction, 11 IEEE Transaction on Man-Machine Systems 190 (1970); *Carbonell & Collins,* Natural Semantics in Artificial Intelligence, Proceedings of the 3rd Int'l Joint Conf. on Artificial Intelligence 344 (1973); *Collins, Warnock & Passafiume,* Analysis and Synthesis of Tutorial Dialogues, in 9 The Psychology of Learning and Motivation (*G. Bower,* ed., 1975); *Brown & Burton,* Multiple Representations of Knowledge for Tutorial Reasoning, in Representation and Understanding 311 (*D. Bobrow & A. Collins,* eds., 1975). *Carbonell's* original system, called SCHOLAR, conducts a tutorial dialogue about the geography of South America, see Carbonell, supra; a later extension of *Carbonell's* work, a system called SOPHIE, conducts a tutorial dialogue about a malfunctioning electronic circuit, see *Brown & Burton,* supra. Both systems are characterized by what is called a mixed initiative interaction: the student can ask questions of the system for a while, and then the system can take charge and ask questions of the student. In an illustrative dialogue from the SOPHIE system, for example the student begins by asking for several voltage measurements in the malfunctioning circuit, and makes a guess about which component is at fault; the computer system then asks the student several questions about the nature of the fault, and discovers that the student does not fully understand the problem. The full dialogue runs as follows (with queries to SOPHIE given in lower case, and responses in upper case):

What is the base emitter voltage of the voltage limiting transistor?
THE BASE EMITTER VOLTAGE OF Q 5 IS –.037 MILLIVOLTS.
What about for the current limiting transistor?
THE BASE EMITTER VOLTAGE OF Q 6 IS .349 VOLTS.
What should it be?
IN A WORKING CIRCUIT THE BASE EMITTER VOLTAGE OF Q 6 IS .593 VOLTS.
Replace Q 6.

But the important question remains: Just how feasible is this extended version of TAXMAN? How likely is it to be constructed in the coming decade? The answer depends both on the characteristics of the problem domain in which we wish to have this extended system operate, and on the expected advances in semantic information processing in the next few years. In order for the current paradigm to be applicable, as we have seen, it is necessary to develop an adequate model of the lower-level factual situations that we expect to occur[164], and an adequate representation of the higher-level legal concepts in the area of the law that we are interested in[165]. But the taxation of corporate reorganizations is an excellent problem domain in both of these respects: it appears that a large number of important and interesting problems can be handled here well before we reach the apparent limits of the TAXMAN paradigm[166]. It is a risky business predicting the future advances in computer science, but based upon a knowledge of both the law of corporate reorganizations and the current work in semantic information processing, I would be willing to hazard a guess. I would make the following highly qualified prediction. If a substantial amount of resources were committed to the project, I would expect that a sophisticated prototype system of demonstrable utility to a corporate tax lawyer could be developed within approximately ten years.

VII. Conclusion

There were two main purposes behind the work on TAXMAN, as set forth in the Introduction to this article, but so far I have dealt explicitly with only one of them. I have described in considerable detail how the current paradigm of semantic

---

I AM GOING TO ASK YOU SOME QUESTIONS ABOUT HOW Q 6 IS FAULTED. ARE ANY JUNCTIONS SHORTED?
I don't know.
ARE ANY TERMINALS OPEN?
Yes.
WHICH TERMINAL(S) ARE OPEN?
The base.
Q 6 IS NOT FAULTED IN THAT MANNER. Q 6 NOT REPLACED.
From this point on, the dialogue becomes even more complex, as the student asks additional hypothetical questions of SOPHIE and SOPHIE responds with selective hints about the source of the problem. See, *Brown & Burton*, supra, at 314–23. *Carbonell's* views on the relative merits of the traditional CAI systems and the more experimental AI systems are well-balanced: "... we are not advocating the complete elimination of [traditional CAI]. It will have its role for some time to come. We see it convenient for cases in which the subject matter is very diversified and the interactions with the students are planned to be brief. In those cases, the development of complex semantic networks is not justified. When discussion in depth is desired, when the student should have some initiative, when detailed anticipation is unwanted, then [AI] systems are to be preferred. On the other hand, when teaching sequences are extremely simple, perhaps trivial, one should consider doing away with the computer, and using other devices or techniques more related to the task." *Carbonell*, supra, at 201.

164 See pp. 360/361 and notes 129-32 supra.

165 See p. 359 and notes 126–28 supra.

166 See note 134 supra.

information processing can be applied to a particular area of the law, the taxation of corporate reorganizations, and I have indicated how this approach might eventually lead to several practical applications. The current version of TAXMAN is capable of producing a slightly simplified analysis of a slightly simplified description of a case like *United States v. Phellis,* but the ideas that are illustrated by this program seem to be applicable to problems of much greater complexity. The TAXMAN paradigm in its extended version, in fact, seems powerful enough to revolutionize the current approaches to computer-aided research and analysis in the law.

But the second purpose – the theoretical one – has not yet been addressed. I suggested earlier that the work on TAXMAN should also provide a tool for the development of our theories about legal reasoning, and should enable us to acquire a more precise understanding of the structure and function and dynamics of legal concepts[167]. I suggested there that we view the TAXMAN system as a formal model of the concepts of corporate reogranization law, and assess its adequacies and inadequacies in this respect[168]. For this second purpose, however, it should be clear by now that there are serious deficiencies in even the extended version of TAXMAN. Many of these have already been noted. The most important deficiencies appeared in the initial discussion of corporate reorganization law in Section II: it was emphasized in that discussion that the "continuity of interest"' "business purpose" and "step transaction" doctrines were developed in sharp opposition to the mechanical manipulation of abstract statutory rules, and that they seemed to have a very different structure from the concepts that have so far been successfully implemented in TAXMAN[169]. A similar observation occurred in the discussion of the feasible extensions of Section VI: although it appeared there that a great many of the rules and concepts of corporate reorganization law could be added to an extended version of TAXMAN, this claim could not be made for all such rules and concepts[170]. A possible explanation of these deficiencies was suggested at the close of Section III, where it was noted that the TAXMAN system was statically equivalent to the higher-order predicate calculus, even though the programming language in which it was written added to this static representation a certain dynamic power and flexibility[171]. In general, then, the detailed analysis of the TAXMAN system has tended to support what would surely be a lawyer's intuition on these matters – that the current TAXMAN paradigm fails to capture the most significant facts about the structure of legal concepts and the process of legal reasoning.

I do not believe, however, that this is the final word on the subject. In Part Two of this article I will explore these theoretical issues in greater detail, assessing more systematically the limitations of the current TAXMAN paradigm, and then suggesting a modified paradigm which appears to correct some of these limitations.

167 See pp. 307/308 supra.
168 See pp. 309/310 supra.
169 See pp. 318–322 supra.
170 See pp. 337 and notes 126–34 supra.
171 See pp. 336/337 supra; see also note 126 supra.

Although I will ultimately come to a conclusion not too dissimilar to the lawyer's intuition – that nothing as complex as legal reasoning could ever be represented in a computer program – I believe it is possible to sketch out a formal computer model that would come somewhat closer to the truth than the current version of TAXMAN. And this, after all, is the task of greatest theoretical interest. Even though our formalisms will always be inadequate in one or more respects, the process of constructing and modifying these formalisms – if carefully done – should itself be a source of insight and understanding[172].

Assistant Professor of Law, State University of New York at Buffalo. B. A. 1966, Yale University; J. D. 1969, Harvard University. The work on this project was begun while the author was a Law and Computer Fellow at the Stanford Law School, 1971–73, and was supported at the time by a grant from the IBM Corporation; additional work was undertaken during a summer workshop at the Stanford Law School in 1975, which was also supported by a grant from the IBM Corporation. Computer time for the project was provided by the Stanford Artificial Intelligence Laboratory. The article in its present form owes a great deal to the suggestions and comments of a number of individuals, including especially *Richard S. Bell, Bruce G. Buchanan, Edward A. Feigenbaum, Marc Galanter, David G. Hays, Thomas E. Headrick, Mary Kay Kane, Jonathan King, Martin E. Lybecker, Ejan Mackaay, Philip Slayton, Colin Tapper* and *George Williams*. None of these individuals is responsible, of course, for any errors of fact or interpretation that remain. Special thanks are due, above all, to Associate Dean *Joseph E. Leininger* of the Stanford Law School, whose support and encouragement over the past few years made this article possible.

172 See pp. 309/310 supra, especially the quotation from *Chomsky* at note 22.